SPECIFICATION-DRIVEN GENERATION AND EVALUATION OF DISCRETE-EVENT WORLD MODELS VIA THE DEVS FORMALISM

Zheyu Chen[1] **Huiteng Zhuang**[2] **Zhuohuan Li**[3] **Chuanhao Li**[3*]

[1]Zhili College, Tsinghua University
[2]School of Transportation Science and Engineering, Beijing University of Aeronautics and Astronautics
[3]Department of Industrial Engineering, Tsinghua University

ABSTRACT

World models are central to LLM agents that must evaluate actions over long horizons. Yet much existing work focuses on environments governed by physical dynamics or spatial structure, whereas many high-impact domains, including supply chains, procurement networks, and business processes, evolve through discrete events, timing constraints, and causal dependencies. These settings call for discrete-event world models. Existing approaches to constructing world models often fall near two extremes: hand-engineered simulators provide consistency and reproducibility, but are costly to build and adapt; neural models are flexible, but can suffer from compounding inconsistency over long-horizon rollouts. We seek a principled middle ground by synthesizing discrete-event world models online from natural-language specifications, retaining the reliability of explicit simulators while gaining the adaptability of neural models. We adopt the DEVS formalism and introduce a staged LLM-based generation pipeline that separates structural inference over component interactions from component-level event and timing logic. For evaluation, we develop benchmark suites in which simulators emit structured event traces, which are then validated against specification-derived temporal, causal, and semantic constraints. This enables reproducible verification and localized diagnostics. Together, these contributions produce world models that remain consistent over long-horizon rollouts, can be verified from observable behavior, and can be synthesized efficiently on demand during online execution.

1 Introduction

LLM agents increasingly need to learn from and plan within complex environments. This makes *world models*, which support hypothetical rollouts under candidate actions, increasingly important. They matter for two reasons. First, agentic reinforcement learning relies on experience-driven scaling, where world models can generate simulated experience to improve learning efficiency [1, 2]. Second, LLM agents are expected to operate over long horizons, where delayed consequences and compounding uncertainty make reliable reasoning difficult; world models let agents simulate downstream effects before acting, supporting more consistent decisions [3, 4]. In novel settings, however, these models cannot always be pre-built and must be synthesized or adapted on demand.

A growing body of work has explored world models across domains [5, 6], but much of it focuses on environments governed by physical dynamics or spatial structure, such as robotic control, visual navigation, and interactive 3D worlds. In contrast, many high-impact real-world systems, including supply chains, procurement networks, service operations, and business processes, are governed by discrete interactions among multiple entities. In these settings, outcomes depend less on continuous physical evolution than on the ordering, timing, and causal dependencies of events. Improving agent performance therefore requires models that capture process structure at the system level, where interventions can yield gains beyond localized physical or spatial optimization [7, 8, 9].

*Corresponding author: chuanhao-li@tsinghua.edu.cn

This motivates the study of *discrete-event world models*: models that represent macro-level environments through state transitions, logical rules, and structured event traces. More broadly, existing methods to constructing world models tend to fall near one of two extremes. At one end, *hand-engineered simulators* explicitly encode environment dynamics through domain rules and process logic. Although reproducible and interpretable, they require substantial human effort and are difficult to adapt online. At the other end, *implicit neural approaches* learn to predict future states directly [3, 10]. Although flexible, they are vulnerable to long-horizon inconsistency. In discrete, process-driven domains, such drift can manifest as invalid event sequences, violated resource constraints, or inconsistent entity states, making purely implicit methods difficult to rely on for consequential planning. Therefore, a valuable middle ground remains underdeveloped: world models that are simultaneously long-horizon consistent, behaviorally verifiable, and synthesizable on demand.

We study this middle ground by formulating world-model construction as the synthesis of executable discrete-event simulators from natural-language environment specifications. The specification provides the domain entities, events, timing assumptions, and causal dependencies, while the generated simulator provides an explicit operational model for rolling out candidate actions. Because the model is executable code, its state updates, event scheduling, and transition logic can be reproduced, inspected, and verified against domain constraints. This preserves the consistency of explicit simulation while reducing the adaptation burden of hand-engineered simulators.

This raises two key challenges. First, realistic environments are structurally complex. Single-pass generation often fails because one localized error can invalidate the full system. Iterative coding agents can compensate through trial-and-error debugging, but those loops incur high token costs and latency, making them unattractive for on-demand synthesis. Second, evaluation is difficult because natural-language specifications are inherently underspecified. One specification may admit multiple valid simulators with different internal abstractions and edge-case behavior. Code-level equivalence is therefore ill-posed, while aggregate KPIs are too coarse: similar KPIs can arise from incorrect event logic, and different KPIs can arise from equally valid interpretations.

To address the first challenge, we adopt the Discrete Event System Specification (DEVS) formalism [11, 12] as the structural scaffold for simulator construction. DEVS prevents logical entanglement by requiring components to interact only through explicit ports. Building on this isolation, we introduce DEVS-GEN, a DEVS model generation pipeline that first plans the system hierarchy and port interfaces, and then synthesizes component logic under these interaction contracts. This decomposition bounds context, enables parallel generation, and avoids costly global debugging loops.

To address the second challenge, we introduce a benchmark based on *trace-based conformance*: evaluating whether the observable behavior of a generated simulator satisfies the temporal, causal, and semantic constraints implied by the specification. Rather than assuming a single canonical implementation, we derive evaluation tasks from high-quality open-source DEVS models whose dynamics provide structured behavioral constraints. The benchmark measures two quantities: *operational success*, which captures whether a simulator respects the required I/O contract and trace schema, and *behavioral conformance*, which captures whether its traces satisfy the expected micro-level state transitions and macro-level causal dependencies. These criteria provide a stable, fine-grained assessment of simulator correctness.

Together, these elements yield a specification-driven framework for generating and rigorously evaluating discrete-event world models. Across seven benchmark scenarios and four model families, our experiments demonstrate two primary advantages: (i), DEVS-GEN exhibits highly predictable efficiency scaling. While iterative agents hit a vertical wall of non-terminating debugging cycles on complex tasks, our decoupled pipeline reliably translates computation time into continuous task completion, scaling to near 100% generation success rate. (ii), ablations reveal a critical architectural trade-off: directly enforcing strict DEVS rules imposes a severe cognitive penalty on LLMs. However, DEVS-GEN successfully absorbs this burden through its explicit planning phase (`PlanTree`), ultimately surpassing the performance of easy-to-write, unconstrained monolithic baselines.

Beyond evaluation, the resulting model hierarchy, port interfaces, and standardized trace schema provide reusable structure for downstream applications. As a proof of concept, we show that automated tools can parse the DEVS topology as a blueprint to systematically instantiate Godot graphical assets, driving real-time frontend animations purely from backend event traces. More broadly, this work supports hybrid systems in which LLMs act as decision-making entities inside event-driven simulators, enabling complex multi-agent processes to be specified, simulated, and inspected from natural-language descriptions [13].

2 Related Works

2.1 World Models: Explicit and Implicit Approaches

World models have long served as predictive mechanisms to support planning and reinforcement learning, enabling agents to reason about the consequences of actions before execution and to generate simulated experience for improved

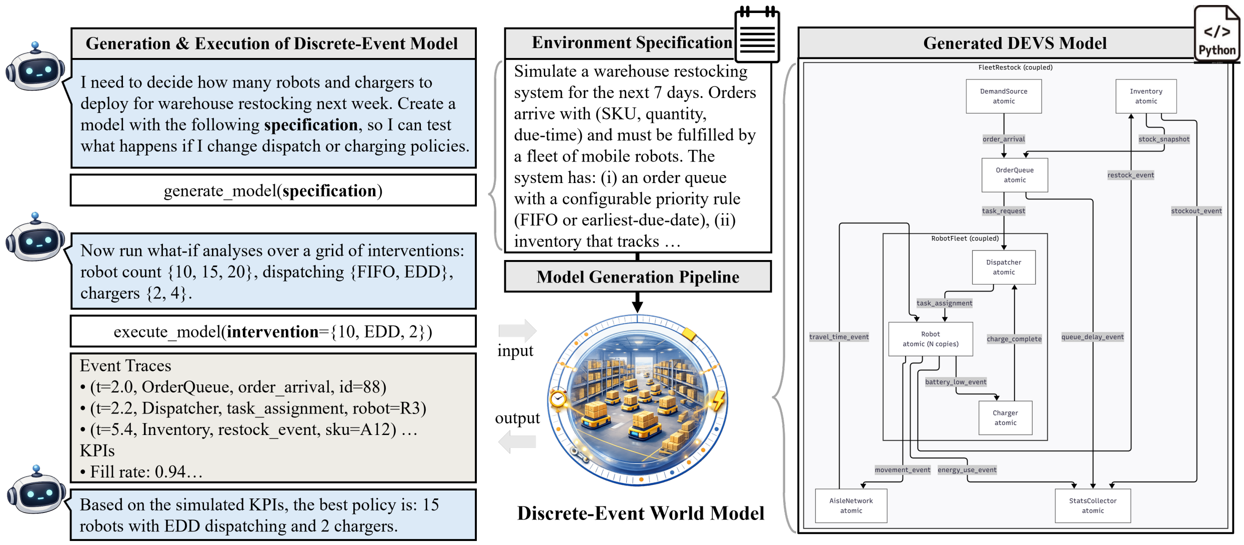

Figure 1: Illustrative example of the generation and execution of a discrete-event world model for a warehouse robot fleet restocking task. To facilitate planning, a natural-language specification is translated into a world model, which exposes a standardized execution interface for interventions (e.g., fleet size, dispatching rule, charger count) and emits a structured event trace and performance metrics. We propose to operationalize such a world model as a DEVS model composed of interacting atomic and coupled components, each representing an entity with local state, event-handling logic, and timing semantics. Interactions among entities are realized through event messages routed via component ports, while event timing is governed by component-specific time-advance and transition functions. This enables systematic what-if analysis and comparative evaluation for online planning.

learning efficiency [1]. Broadly, existing work on world modeling can be categorized into two complementary approaches: *explicit world models*, in which environment dynamics are manually implemented inside executable simulators, and *implicit world models*, in which dynamics are captured by learned models, often neural networks.

Several works construct world models as human-designed, executable simulators with well-defined state representations and transition dynamics. For example, [14] utilize a robotic simulator as a world model and demonstrate that interaction with such a model can enhance the performance of language models on downstream tasks. Similarly, [6] present *Web World Models*, a carefully web-based simulator that captures the structure and dynamics of web environments to support controlled agent interaction and evaluation. These approaches emphasize executability, reproducibility, and precise control over environment dynamics. In parallel, recent work has explored whether environment dynamics can be modeled implicitly by large neural networks, particularly LLMs. In this paradigm, the world model is not represented as a standalone simulator; instead, environment dynamics are encoded in the model's parameters and accessed through prediction during inference. LLMs are prompted or trained to predict future observations or outcomes conditioned on past events, and these predictions are used to guide action selection or planning [3, 4]. This approach has been applied across a range of domains, including web environments [4, 6, 15], video games and real-world video data [5], and robotic tasks involving visual, force, or other sensory feedback [16].

These two lines of work reflect different trade-offs in how world dynamics are represented and accessed. Explicit world models provide executable, transparent representations of environment dynamics but typically require substantial manual effort to design and adapt, while implicit neural world models offer greater flexibility and scalability by leveraging learned representations, at the cost of reduced explicitness and multi-turn consistency.

2.2 Symbolic and Programmatic World Model Generation

A closely related line of work focuses on generating structured representations of environment dynamics from natural language, with the goal of improving interpretability and enabling rule-based or formal verification. In the context of symbolic world or domain models, several approaches translate natural-language descriptions into executable planning representations such as PDDL, using plan execution behavior or task outcomes as evaluation signals [17, 18, 19]. Complementary efforts reduce reliance on expert supervision by incorporating environment interaction to iteratively refine generated planning domains [20].

Beyond planning-oriented representations, related work has explored generating system-level models from natural-language requirements in software and systems engineering. In particular, several studies synthesize SysML artifacts, such as structural diagrams or behavioral models, and evaluate them using consistency checks, requirement coverage, or simulation-based criteria [21, 22, 23]. These approaches emphasize requirements traceability, design validation, and alignment with established engineering workflows, but typically focus on static structure or high-level behavioral descriptions rather than executable environment dynamics.

Our work aligns with these efforts in seeking explicit, programmatic representations of environment dynamics from natural language, but differs in its choice of modeling formalism and evaluation focus. Compared to PDDL-based planning models, which emphasize symbolic state transitions and goal satisfaction in discrete steps, DEVS provides an execution-oriented semantics centered on discrete events, explicit time, and concurrent interactions, making it well suited for domains such as queues, workflows, distributed systems, and message-driven multi-agent settings. Relative to SysML-based approaches, which primarily support high-level system specification and design reasoning, DEVS offers precise simulation semantics and inherent executability, producing structured event traces that enable black-box, trace-based evaluation under controlled scenarios. Together, these properties position DEVS as a natural foundation for synthesizing and evaluating discrete-event world models from natural-language descriptions.

2.3 LLM-Based Code Generation and Long-Horizon Development

Recent research has extensively explored the capabilities of large language models and code agents for natural language driven software implementation, fixing issues in existing code repositories, and generating multi-file projects. Evaluations typically focus on unit-test-based functional correctness, issue-fixing success rates, or repository-level benchmarks. Examples include unit-test-driven functional correctness evaluation [24], repository-repair benchmarks based on real GitHub issues [25], and repository-level multi-file evaluation settings [26, 27].

The problem addressed in this paper likewise involves long-horizon code development: coordinating across multiple steps and files, and leveraging execution and tool feedback to iteratively advance the implementation. Existing work often summarizes complex processes with automatable, reproducible outcome metrics (e.g., test pass/fail or task completion rate) and evaluates systems on multi-step interactive software-engineering tasks or repository-level feature-implementation settings [28, 27, 29].

Our evaluation objectives differ fundamentally from these approaches. We require not only executable code but also conformance to a time-sensitive event-semantics specification, defined via standard-input interventions and a unified JSONL event format. Accordingly, our evaluation is closer to conformance testing, where correctness is judged by whether observable input/output behavior satisfies the specification, rather than by generic unit tests [30]. It also aligns with the runtime-verification perspective of checking specifications against execution traces [31].

2.4 Verification, Validation, and Conformance for Discrete-Event Models

Verification and validation (V&V) of discrete-event models and DEVS models have a long research history, including static consistency checks, simulation-based verification, and formal analysis of model composition. These approaches provide systematic tools for detecting specification violations. For instance, structural consistency verification can validate the structural definitions of coupled models prior to simulation [32]; behavioral-level verification is often instantiated as simulation-driven unit tests or semantic-consistency tests [33, 34]; and work that integrates DEVS with model checking and applies mixed V&V in hierarchical/multi-resolution compositional modeling illustrates a path that combines formal analysis with compositional verification [35, 36].

In line with this tradition, execution trajectories can serve as a direct basis for systematic evaluation: given input drivers and observation schemes, the runtime traces of the system under test can be compared against specified properties to determine whether the expected behavior is satisfied. This perspective is commonly framed as runtime verification (RV) in the broader software and systems verification literature, where specifications are checked online or offline against event/state sequences [31]. In discrete-event simulation, similar trace-based checking can be achieved by explicitly modeling properties and performing on-demand checks during simulation [37].

This paper similarly adopts trace-based system evaluation, but under a distinct context: the DEVS models are automatically synthesized by LLM-driven generators, resulting in highly diverse implementation forms. Consequently, the evaluation framework must center on interfaces and observable behaviors to accommodate diverse implementations, and provide actionable localization signals when specification violations are detected. Furthermore, evaluation should target property/behavioral conformance rather than relying on code-level equivalence to a single reference implementation. These requirements naturally connect to black-box testing frameworks and their diagnostic capabilities for failing cases

[38]. They also echo metamorphic testing, which mitigates the oracle problem via metamorphic relations when reliable test oracles are unavailable [39].

3 Discrete-Event World Models and DEVS

In this section, we formalize the class of discrete-event world models studied in this paper and then introduce DEVS as the operational representation we use to instantiate them. Our goal is to characterize (i) the scope of environments we target, (ii) the input-output interface exposed by an executable world model, and (iii) why DEVS is a suitable representation for this setting.

3.1 Problem Setup

We study the synthesis of *executable discrete-event world models* directly from *natural-language specifications*. These specifications informally describe the environment's interacting entities, permissible discrete events, and their causal or temporal relationships (see App. F.1 for an example specification). Figure 1 illustrates this pipeline using a warehouse robot restocking task, where the input text describes entities such as robots and chargers together with events such as task assignment and charging. Although the figure illustrates how the pipeline can be integrated into an agent's reasoning loop to facilitate online planning, it can also be used offline to construct a standalone environment.

The *discrete-event world* abstraction targets environments whose decision-relevant behavior is driven by event timing, resource constraints, and multi-entity interaction rather than fine-grained continuous physics. This includes settings such as service operations, supply chains, business processes, and distributed workflows, where agents must reason about queues, delays, synchronization, and shared resources. Related formalisms can encode some aspects of these settings, but they organize them around different primary abstractions. MDP-style models foreground state, action, and reward for sequential decision making, while PDDL-based representations emphasize symbolic action preconditions, effects, and goal satisfaction [17, 18, 19, 20]. In contrast, we focus on *executable simulators* with explicit temporal semantics. This is important for environments with asynchronous exogenous events, overlapping activities, and contention for shared resources, such as jobs waiting for a server or robots competing for chargers. These patterns are natural in discrete-event models but require substantial extra encoding in MDP or PDDL formulations. We therefore adopt a discrete-event perspective centered on event timing, component interaction, and observable trajectories.

Discrete-Event World. A discrete-event world is an environment where the state changes only at isolated event times, while remaining constant between these events. In the warehouse example, the world state includes, for instance, the contents of the order queue, the locations and battery levels of robots, and inventory levels. State changes occur only when events such as an order arrival, a robot completing a task, or a charging operation finishing take place. Formally, a discrete-event world is represented as $\mathcal{W} = (\mathcal{E}, \mathcal{S}, \Omega, \mathcal{P}, \delta)$. $\mathcal{E}$ is a finite set of entities encompassing elements like order queues, robots, and inventory. The world state space is denoted by $\mathcal{S} = \prod_{e \in \mathcal{E}} \mathcal{S}_e$, where $\mathcal{S}_e$ is the local state space of an individual entity e. The event alphabet Ω specifies the available event types, and $\mathcal{P}$ defines the payload space that captures event-specific data, e.g., order IDs or robot identifiers.

An event instance $u = (\ell, p) \in \mathcal{U}$ represents the occurrence of an event, e.g., a specific robot completing a designated task, where $\mathcal{U} := \Omega \times \mathcal{P}$ is the space of all such instances. The transition function $\delta : \mathcal{S} \times (\mathcal{U} \cup \{\varnothing\}) \times \mathbb{R}_{\geq 0} \to \mathcal{S}$ defines the system dynamics: given a state $s \in \mathcal{S}$, an event instance u or $\varnothing$, and a nonnegative time increment $t \in \mathbb{R}_{\geq 0}$, it returns the resulting state $\delta(s, \varnothing, t)$ or $\delta(s, u, t)$. The former represents state evolution driven solely by time, while transitions associated with u capture interactions among entities, e.g., a dispatcher assigning a task to a robot.

World Models as Simulators. While $\mathcal{W}$ defines the abstract rules of the environment, an autonomous agent requires a concrete, interactable software artifact. Therefore, in our framework, an executable discrete-event world model is realized as a simulator, denoted as $\mathcal{M}$. It instantiates a concrete interpretation of the world $\mathcal{W}$, operationalizes its event dynamics, and serves as a predictive substrate for downstream tasks as depicted on the left side of Figure 1. The simulator is treated as a black-box executable with a fixed input–output interface specification. External interventions are drawn from a set $\mathcal{A}$ and provided as configuration arguments

$$\mathcal{I} = (a_1, a_2, \ldots, a_m), \qquad a_k \in \mathcal{A},$$

which parameterize execution at launch. In the warehouse example, these interventions include the number of robots, the choice of dispatching policy, the number of chargers, and the simulation horizon. Some tasks additionally provide an optional input stream $\mathcal{J}$ specifying exogenous inputs delivered during execution. We treat the overall external input to the simulator as the pair $(\mathcal{I}, \mathcal{J})$. Executing $\mathcal{M}(\mathcal{I}, \mathcal{J})$ produces observable behavior in the form of an event trace $\mathcal{T} = (r_1, r_2, \ldots)$ with each record $r = (t, e, \ell, p) \in \mathbb{R}_{\geq 0} \times \mathcal{E} \times \Omega \times \mathcal{P}$ being a time-stamped output, This could correspond to an order arrival or a robot completing a charging operation at time t.

World Model Synthesis Task. The task is to generate a simulator $\mathcal{M}$ from a possibly under-specified Spec of a discrete-event world $\mathcal{W}$. Spec encompasses two dimensions: (1) Operational configurations $\texttt{Spec}_{\mathrm{ope}}$ that define the interaction boundaries $(\mathcal{I}, \mathcal{J})$ and the structured trace format $\mathcal{T}$; and (2) Behavioral descriptions $\texttt{Spec}_{\mathrm{beh}}$ that specify the causal and temporal dynamics of $\mathcal{W}$ in natural language. The strictness of Spec depends on the application context. Real-world applications often feature informal or partially implicit Spec, allowing the synthesis pipeline to infer reasonable simulation defaults. In contrast, for automated benchmarking, we adopt a highly specified regime in which $\mathcal{I}$, $\mathcal{J}$, and $\mathcal{T}$ are governed by a strict schema. This makes the benchmark more challenging and meaningful: a looser Spec imposes fewer constraints on the generated simulator $\mathcal{M}$ and thus makes validation easier, whereas a stricter Spec yields richer behavioral obligations and more discriminative checks. Our framework is flexible enough for diverse real-world uses while supporting rigorous, automated evaluation under controlled benchmark conditions.

3.2 DEVS as an Operational Representation

The discrete-event world abstraction above does not commit us to one implementation formalism. In principle, such environments could be realized using other executable representations, including domain-specific simulators, Petri nets, queueing networks, or other explicit state-transition models. We choose Parallel DEVS (PDEVS) [12] because it provides a modular and hierarchical executable semantics for the event timing and concurrent interactions, together with a clean separation between local component behavior and global interaction structur. The formal definition of PDEVS is provided in App. B. For brevity, we refer to PDEVS simply as DEVS throughout the remainder of the paper. Concretely, DEVS realizes this modular structure through two fundamental component types:

Atomic Models (Behavioral Logic). Atomic models encapsulate the local state, timing semantics, and event-handling rules of individual entities, e.g., robots or order queues. They autonomously update local state and emit events in response to elapsed simulated time or incoming events.

Coupled Models (Structural Routing). Coupled models define the system architecture by composing sub-components and specifying how events are routed among their input and output ports. They carry no local state or behavioral logic; instead, they make the communication structure explicit.

By recursively assembling these components, a hierarchical world model $\mathcal{M}$ is formed. Complex macro-level dynamics, e.g., a dispatcher assigning tasks to multiple robots, emerge from standardized event passing across this explicitly defined topology. In the next section, we leverage this structure to separate global planning of component interactions from local synthesis of component behavior.

4 DEVS-Based Model Generation Pipeline

Building on the DEVS formalization in Section 3, we introduce DEVS-GEN, a pipeline that synthesizes a specification Spec into an executable discrete-event simulator $\mathcal{M}$. The key idea is to use DEVS as a structural scaffold: rather than generating the simulator monolithically, the pipeline decomposes synthesis into a sequence of well-scoped tasks. The prompts used in DEVS-GEN are provided in App. D.

As formalized in Algorithm 1, DEVS-GEN constructs $\mathcal{M}$ in two stages. First, *structural planning* derives the architectural PlanTree from Spec. Second, *behavioral synthesizing* implements each component under its assigned ModelPlan and assembles the resulting modules into the final simulator. The upper half of Figure 2 illustrates this procedure using the warehouse robot fleet restocking example.

The central intermediate representation connecting the two stages is the PlanTree, a hierarchical blueprint that dictates the simulator's topology, assigns model types (coupled or atomic), and defines port-based interaction contracts between models. Each node corresponds to an environment component, such as a robot, charger, or order queue, and is associated with a component-specific ModelPlan. This serves as a local contract, specifying the node's model type, input/output port schemas, state variables, and event and timing logic, and, for coupled models, the interaction graph over its child components.

All functional operations in the pipeline (e.g., structural drafting, interface routing, and code synthesis) are operationalized via specific LLM prompt templates. The exact prompts mapping to the pseudocode functions are comprehensively detailed in Appendix D.

4.1 Stage 1: Structural Planning

Stage 1 transforms Spec into PlanTree. To control complexity, this stage is decomposed into the following two steps.

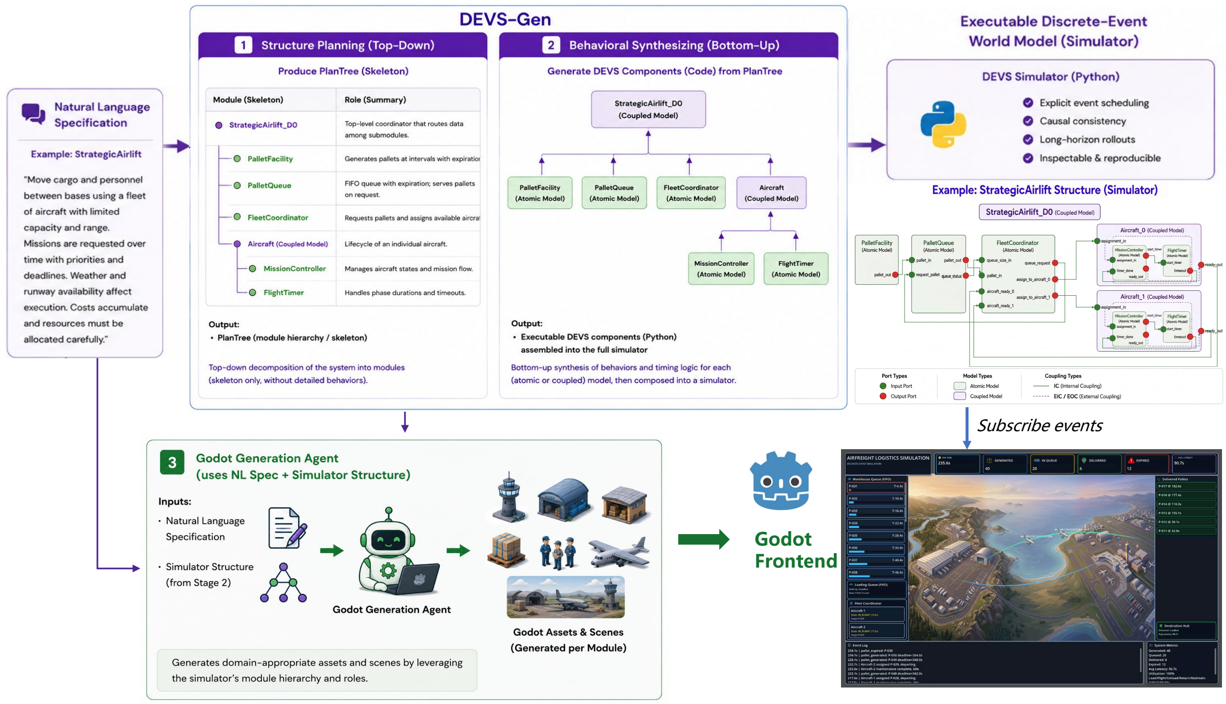

Figure 2: Illustration of DEVS-GEN. A natural-language specification is transformed into a discrete-event world model via a two-stage pipeline; its generated artifacts, including the model hierarchy and event stream, can be used to build visualization frontends for animation and interactive inspection.

Algorithm 1 DEVS-GEN Pseudocode

Input: specification `Spec`
Output: simulator $\mathcal{M}$ (DEVS coupled model)
`PlanTree` $\leftarrow$ PLAN(`Spec`) *// See Alg. 2*
$\mathcal{M}$ $\leftarrow$ CONSTRUCT(`PlanTree`) *// See Alg. 3*

System Skeleton Drafting. Given only `Spec` as input (see App. F.1), the pipeline first extracts the system's high-level structure as a `Skeleton` (see App. F.2). This artifact captures the global topology by identifying the required modules, organizing coupled models into a parent-child hierarchy, and assigning each entity a concise functional description. The resulting skeleton provides a shared architectural frame that anchors all subsequent steps.

Architecture Refinement and Sub-tasks. Guided by the `Skeleton`, which is a tree of different nodes, the pipeline executes a top-down recursive refinement to expand each bare node into a detailed `ModelPlan` (Algorithm 2, Step 2). When visiting a coupled model, the planner first resolves its internal port-to-port event routing and explicitly assigns preliminary interface contracts to its children. Because these child boundaries are now fixed by the parent, the algorithm can recursively refine all child nodes in parallel. When the traversal reaches an atomic model, the refinement process finalizes the local state variables, event payloads, and timing semantics required for execution.

The relationship among these artifacts is purely structural: `PlanTree` is exactly the `Skeleton` fully enriched with `ModelPlans`. Once every node in the initial skeletal hierarchy is populated with its interface contracts and local responsibilities, the artifact becomes the final `PlanTree` (visualized in App. F.3). By fixing these boundaries upfront, Stage 1 strictly bounds each component, allowing Stage 2 to synthesize executable code for different components in parallel and in isolation. This reduces long-context failure modes and yields code that is easier to verify and assemble.

Algorithm 2 STAGE 1: STRUCTURAL PLANNING

```
function PLAN(Spec)
    // Step 1: Global skeleton inference
    Skeleton ← INFERGLOBALSKELETON(Spec) // via Prompt in App. D.1
    PlanTree← INITIALIZETREEHIERARCHY(Skeleton) // initialize the tree structure

    // Step 2: Architecture refinement (Top-Down Parallel)
    REFINENODE(Spec, PlanTree.root)

    return PlanTree
end function
function REFINENODE(Spec, v)
    if v is Coupled Model then
        REFINECOUPLEDPLAN(Spec, v) // via Prompt in App. D.1
        // Refine children in parallel once their contracts are set
        for all child ∈ v.children in parallel do
            REFINENODE(Spec, child)
        end for
    else if v is Atomic Model then
        v.ModelPlan ← REFINEATOMICPLAN(Spec, v) // via Prompt in App. D.1
    end if
end function
```

4.2 Stage 2: Behavioral Synthesizing

Stage 2 transforms the hierarchical `PlanTree` into the final executable simulator $\mathcal{M}$. This is achieved by synthesizing components as independent Python modules based on the `xdevs.py` framework [12] and systematically assembling them into a cohesive project.

Parallel Component Synthesis. Because Stage 1 explicitly resolves all inter-component routing dependencies via port interfaces, sibling components within any coupled model share no implementation-level data dependencies. This structural decoupling allows Stage 2 to synthesize these sibling modules entirely in parallel (Algorithm 3, Line 6). Governed by its specific `ModelPlan` and shared `xdevs.py` syntax constraints, each component is generated as a discrete file. For atomic models, the LLM implements local state transitions, event handlers, and trace emissions. For coupled models, the LLM generates the structural logic to instantiate children and wire their ports.

Bottom-Up Interface Reconciliation. To ensure seamless integration across these modules, we adopt a bottom-up assembly strategy. Although each child component strictly follows its `ModelPlan`, LLMs may occasionally introduce minor variations in variable naming or method signatures. To prevent these localized typos from cascading into integration failures, DEVS-GEN extracts the *as-implemented* interface summaries directly from the children's newly generated source code. The parent coupled model is then synthesized conditioned on both its original `ModelPlan` and these concrete child summaries, guaranteeing perfect linkage.

Assembly and Execution Entry. Because the DEVS formalism strictly enforces modularity, the physical assembly of $\mathcal{M}$ requires no complex code merging; it simply involves organizing the independently generated, file-level modules into a standard Python package. Finally, the pipeline prompts the LLM to synthesize a top-level execution script. This entry point instantiates the root coupled model, applies the external interventions $\mathcal{I}$, and launches the simulation environment.

4.3 Automated Visualization of Generated World Models

Beyond synthesis and evaluation, the outputs of DEVS-GEN support a repeatable automated visualization procedure. As illustrated in Figure 2, given a generated `PlanTree` and the original problem description, a generic coding agent equipped with a Godot-development skill[2] can automatically construct a 2D or 3D visualization frontend. The agent is additionally instructed to subscribe to the simulator's event trace stream via MQTT so that the scene reacts to runtime events emitted by $\mathcal{M}$.

[2] `https://github.com/htdt/godogen`

Algorithm 3 STAGE 2: BEHAVIORAL SYNTHESIZING

function CONSTRUCT(`PlanTree`)
  $\mathcal{M} \leftarrow \emptyset$
  `AllSummary` $\leftarrow$ []
  for all $child \in$ `PlanTree`.$children$ **in parallel do**
    $\mathcal{M} \leftarrow \mathcal{M} \cup$ CONSTRUCT($child$)
    `AllSummary`.append(SUMMARIZE($\mathcal{M}$)) *// via Summarizer Prompt, App. D.2*
  end for
  // Code synthesis via Main Code Gen Prompt, App. D.2
  $\mathcal{M} \leftarrow \mathcal{M} \cup \{$ generate code using `PlanTree`, `AllSummary`$\}$
  // Final assembly: generate entry point for the root model
  if v is `PlanTree`.$root$ **then**
    $\mathcal{M} \leftarrow \mathcal{M} \cup \{$ generate top-level execution script for v $\}$
  end if
  return $\mathcal{M}$
end function

This procedure is practical because DEVS-GEN exposes the simulator through reusable intermediate artifacts rather than only as source code. The `PlanTree` provides an explicit scene blueprint, including the environment hierarchy and component roles, while the standardized event trace gives a stable interface for driving animations from live execution. As a result, the visualization agent does not need to reverse-engineer the generated simulator implementation; it only maps `PlanTree` entities to scene objects and event types to animation routines. This makes visualization a lightweight extension of the pipeline and a useful feature for human inspection, demo construction, debugging concurrent interactions, and future multimodal agent interfaces built on top of the same event stream.

5 Trace-Based Evaluation of World Models

The world model synthesis task requires an agent to generate a valid simulator $\mathcal{M}$ from a `Spec`. To evaluate this capability, we use a benchmark grounded in *trace-based conformance*: whether the emitted event traces $\mathcal{T}$ satisfy the dynamic, temporal, and causal constraints described in the `Spec`. This yields a fine-grained, implementation-agnostic measure of simulator correctness.

5.1 Trace-Based Evaluation Criteria

Our benchmark evaluates the generated simulator $\mathcal{M}$ against the two dimensions of $\texttt{Spec} = (\texttt{Spec}_{\mathrm{ope}}, \texttt{Spec}_{\mathrm{beh}})$. Concretely, we translate $\texttt{Spec}_{\mathrm{ope}}$ and $\texttt{Spec}_{\mathrm{beh}}$ into executable testing rules.

Operational Success. The simulator must execute without runtime errors and adhere to the I/O contract, accepting interventions $\mathcal{I}$ and emitting structured traces $\mathcal{T}$ in the prescribed schema.

Behavioral Conformance. For simulators that achieve operational success, we further evaluate whether their observable behavior conforms to $\texttt{Spec}_{\mathrm{beh}}$. We implement rules at two granularities: (i)*Micro-level consistency*: checks on the state evolution of individual components. For example, in a queueing system, a rule verifies whether service times and waiting intervals follow the timing logic in the behavioral description. (ii)*Macro-level causality*: checks on causal and temporal relationships across interacting components. These rules detect failures such as events occurring without proper coupling, effects appearing before causes, or deviations from prescribed statistical behavior in stochastic settings.

5.2 Dataset Construction and Composition

To evaluate these criteria systematically, we construct a curated benchmark dataset. More details about the dataset are provided in App. E. Rather than inventing rules by hand, we select high-quality open-source DEVS models from diverse domains, such as network protocols and industrial supply chains. We transcribe their core dynamics into the natural-language behavioral descriptions $\texttt{Spec}_{\mathrm{beh}}$, design the corresponding $\texttt{Spec}_{\mathrm{ope}}$ and checker scripts. Crucially, we enforce a highly strict specification regime to isolate pure code synthesis capability from requirement elicitation, preventing loose constraints from artificially inflating validation scores.

Data Assets. Each benchmark scenario is a self-contained evaluation package containing: (i) Target `Spec`: the input to the generation pipeline, partitioned into $\texttt{Spec}_{\mathrm{ope}}$ and $\texttt{Spec}_{\mathrm{beh}}$; (ii) Test suite (multiple $\mathcal{I}$): a set of inputs designed to trigger specific causal dependencies and stress-test the generated simulator under varying conditions; (iii) Checker Scripts: checkers that compute the operational and behavioral scores.

Dataset Composition. Table 1 summarizes the benchmark scenarios. The dataset is stratified along two axes. First, by *domain semantics*, it covers service operations, network protocols, and physical dynamics. Second, by *dynamic features*, it spans stochastic queueing, schedule-driven delays, continuous dynamics via ODE approximations, and complex nested state machines.

Table 1: Benchmark scenarios summary. The dataset covers diverse domains and dynamic characteristics, ordered by their overall scale. We further quantify complexity through specification length (Words), number of entities (Ents), distinct event types (Evts), and reference code size (Ref. LOC).

Scenario	Domain	Key Dynamics	Words	Ents	Evts	Ref. LOC	Scale
SEIRD	Bio-Math	Continuous (ODE)	507	5	1	745	S
ABP	Network	Stop-and-Wait	854	4	5	573	S
Barbershop	Service	Blocking & Signals	700	3	2	570	S
IOBS	Banking	Pipeline w/ Routing	822	6	7	703	M
OTrain	Transport	Schedule-driven	755	4	4	961	M
FileTransfer	Network	Dual-Loop FSM	750	8	12	937	L
StratAirlift	Logistics	Active Reneging	966	5	9	1476	L

5.3 Metrics and Experimental Protocol

To assess the quality of a simulator $\mathcal{M}$ for a scenario, we evaluate it on a test suite $\mathcal{D}$ consisting of N test cases $\{d_1, d_2, \ldots, d_N\}$. Each test case d_i provides a specific input $\mathcal{I}_i$ to the simulator, resulting in an emitted event trace $\mathcal{T}_i = \mathcal{M}(\mathcal{I}_i)$. We summarize performance with two metrics.

Operational Score ($\text{Score}_{\text{ope}}$**).** This metric quantifies adherence to $\texttt{Spec}_{\text{ope}}$. For each test case i, we define a binary validity indicator $v_i \in \{0, 1\}$: $v_i = \mathbb{I}(\text{ExitCode} = 0) \cdot \mathbb{I}(\text{NoTimeout}) \cdot \mathbb{I}(\text{ValidSchema}(\mathcal{T}_i))$, i.e., $v_i = 1$ if and only if the simulator executes without runtime errors and emits an event trace that conforms to the schema. The operational score $\text{Score}_{\text{ope}} = \frac{1}{N}\sum_{i=1}^{N} v_i$.

Behavioral Score ($\text{Score}_{\text{beh}}$**).** This metric evaluates fidelity to $\texttt{Spec}_{\text{beh}}$. Let $\mathcal{R}_{\text{micro}}$ denote the set of micro-level consistency rules and $\mathcal{R}_{\text{macro}}$ the set of macro-level causality rules. To balance these two granularities, we compute a macro-average of their pass rates. Each rule r is a binary evaluation function: $r(\mathcal{T}) = 1$ if trace $\mathcal{T}$ satisfies the checked constraint, and 0 otherwise.

For a given test case i, the behavioral conformance score c_i is calculated as:

$$c_i = \frac{1}{2} v_i \left(\underbrace{\frac{\sum_{r \in \mathcal{R}_{\text{micro}}} r(\mathcal{T}_i)}{|\mathcal{R}_{\text{micro}}|}}_{\text{Micro-level Consistency}} + \underbrace{\frac{\sum_{r \in \mathcal{R}_{\text{macro}}} r(\mathcal{T}_i)}{|\mathcal{R}_{\text{macro}}|}}_{\text{Macro-level Causality}} \right) \tag{1}$$

Note the calculation includes v_i, which means if the simulator fails operationally, its behavioral score is 0. The behavioral score is $\text{Score}_{\text{beh}} = \frac{1}{N}\sum_{i=1}^{N} c_i$.

6 Experiments

In this section, we evaluate the DEVS-GEN and several LLM-based software engineering agents. Our evaluation addresses three questions: (i) *Effectiveness:* Can DEVS-GEN reliably synthesize correct world models? (ii) *Efficiency:* How does the resource consumption (time and tokens) compare to iterative agents? (iii) *Ablation:* How do strict DEVS formalism constraints impact the generation process, and how does our pipeline mitigate these challenges?

6.1 Experimental Setup

We conduct experiments using the benchmark we built in Section 5. All local environments are executed on a CPU machine, interfacing with LLMs via API endpoints like OpenRouter. We report the $\text{Score}_{\text{ope}}$ and $\text{Score}_{\text{beh}}$. For efficiency,

Table 2: Main Results on Effectiveness. We group the frameworks into *Iterative* (utilize execution feedback to debug) and *Single-Pass* (generate models directly). **Bold** indicates the best performance within each respective group. DEVS-GEN function best for single-pass generation and achieves performance highly comparable to fully iterative agents.

		Single-Pass (No Execution)			Iterative (w/ Execution)	
Model	**Metric**	**DEVS-GEN (Ours)**	**OpenH-L**	**SWE-L**	**OpenH**	**SWE**
GPT-5.2	$\text{Score}_{\text{beh}}$	0.87 ± 0.27	$\mathbf{0.91 \pm 0.23}$	0.00 ± 0.00	0.90 ± 0.24	0.82 ± 0.36
	$\text{Score}_{\text{ope}}$	0.90 ± 0.20	$\mathbf{0.95 \pm 0.22}$	0.00 ± 0.00	$\mathbf{0.95 \pm 0.22}$	0.86 ± 0.36
GLM-4.7	$\text{Score}_{\text{beh}}$	$\mathbf{0.75 \pm 0.23}$	$\mathbf{0.75 \pm 0.27}$	0.56 ± 0.42	$\mathbf{0.91 \pm 0.09}$	0.70 ± 0.41
	$\text{Score}_{\text{ope}}$	$\mathbf{0.95 \pm 0.24}$	0.90 ± 0.30	0.76 ± 0.44	$\mathbf{1.00 \pm 0.00}$	0.76 ± 0.44
Qwen3-30B	$\text{Score}_{\text{beh}}$	0.59 ± 0.32	0.60 ± 0.38	$\mathbf{0.66 \pm 0.24}$	0.61 ± 0.38	$\mathbf{0.73 \pm 0.29}$
	$\text{Score}_{\text{ope}}$	0.82 ± 0.39	0.76 ± 0.44	$\mathbf{0.95 \pm 0.22}$	0.86 ± 0.35	$\mathbf{0.90 \pm 0.30}$
Llama-4-17B	$\text{Score}_{\text{beh}}$	$\mathbf{0.31 \pm 0.27}$	0.22 ± 0.30	0.24 ± 0.25	0.12 ± 0.23	$\mathbf{0.28 \pm 0.30}$
	$\text{Score}_{\text{ope}}$	$\mathbf{1.0 \pm 0}$	0.43 ± 0.51	0.67 ± 0.48	0.28 ± 0.45	$\mathbf{0.57 \pm 0.51}$

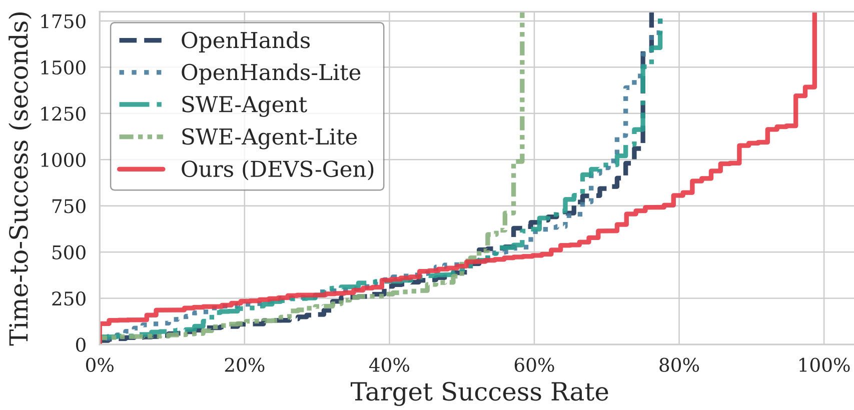

Figure 3: Time-to-Success distribution across all evaluated models. The x-axis represents the cumulative success rate, and the y-axis indicates the wall-clock time required to achieve it. While baseline agents show low initial latency, they hit a vertical wall before reaching an 80% success rate due to non-terminating debugging cycles. In contrast, DEVS-GEN translates computation time into continuous task completion, reliably scaling to near 100% success.

while token tracking can vary across different agent runtimes, so we use wall-clock time as a universal indicator of debugging stagnation.

For the main effectiveness evaluation, we compare DEVS-GEN against two open-source agentic frameworks: `OpenHands` [40] and `SWE-Agent` [28]. We evaluate their Standard (Iterative) configs alongside their restricted Lite (Non-Iterative) variants to isolate the impact of execution feedback.

6.2 Main Results

Effectiveness. Table 2 reports the mean and standard deviation for $\text{Score}_{\text{ope}}$ and $\text{Score}_{\text{beh}}$. Across all models, DEVS-GEN achieves operational success rates comparable to fully iterative agents. It accomplishes this while operating entirely without code execution or debugging loops, suggesting that the structural blueprint provided by `PlanTree` acts as a "correct-by-construction" guide.

The utility of our specification-driven approach is further highlighted when compared to the non-iterative baselines. Without execution feedback, unconstrained agents are highly susceptible to hallucination. By anchoring the generation to `PlanTree`, DEVS-GEN maintains stable effectiveness in a single pass.

Efficiency. Relying on metrics like average execution time to evaluate efficiency introduces severe bias, as the timeouts are not properly reflected. So we presents the Time-to-Success distribution in Figure 3, illustrating the time cost required to achieve a target success rate over all tested models.

The distribution reveals a critical bottleneck for standard agents. While frameworks like `OpenHands` and `SWE-Agent` exhibit competitive latency on some simple tasks, they rapidly hit a vertical wall. They fall into non-terminating debugging cycles and fail to surpass an 80% success threshold, regardless of the time budget.

In contrast, DEVS-GEN demonstrates highly predictable scaling. Although constructing the `PlanTree` incurs a minor initial overhead, this structural decoupling enables parallel synthesis. It translates computation time into continuous task completion, pushing its success rate toward 100%.

6.3 Ablation: Framework Constraints and Cognitive Load

The DEVS formalism provides a modular architecture for complex simulators, but directly enforcing its strict structural rules imposes a heavy cognitive load on LLMs. To evaluate this, we compare three non-iterative setups: unconstrained direct prompting `single` and constraint-forced direct prompting `single-xdevs` (forced to use `xdevs.py`, equipped with necessary knowledge of the framework) to establish the baseline cognitive load of the DEVS formalism, comparing them against DEVS-GEN.

Table 3 reports the performance on a representative Large (GLM-4.7) and Small (Qwen3-30B) model. LLMs achieve robust baseline scores when generating free-form, monolithic scripts (`single`). However, enforcing highly structured DEVS semantics (`single-xdevs`) causes significant drops. This indicates that simultaneously managing global architectural routing and local formalism syntax overwhelms the model's context capacity.

DEVS-GEN mitigates this structural complexity. By pre-solving the architectural routing and assigning interface contracts via `PlanTree`, the pipeline allows the LLM to focus entirely on localized logic. Consequently, DEVS-GEN overcomes the cognitive penalty of the formal constraints and surpasses the unconstrained baselines (e.g., improving $\text{Score}_{\text{ope}}$ to 0.95 on GLM-4.7).

Table 3: Ablation on Framework Constraints. We report $\text{Score}_{\text{ope}}$ and $\text{Score}_{\text{beh}}$. Directly enforcing strict structural rules (`single-xdevs`) causes a performance drop across both metrics compared to free-form coding (`single`). DEVS-GEN utilizes the `PlanTree` scaffold to absorb this complexity, surpassing the unconstrained baselines.

	GLM-4.7 (Large)		**Qwen3-30B (Small)**	
Setup	**$\text{Score}_{\text{ope}}$ ↑**	**$\text{Score}_{\text{beh}}$ ↑**	**$\text{Score}_{\text{ope}}$ ↑**	**$\text{Score}_{\text{beh}}$ ↑**
`single` (Unconstrained)	0.74	0.57	0.74	0.51
`single-xdevs` (Strict `xdevs`)	0.45 (↓ 0.29)	0.30 (↓ 0.27)	0.66 (↓ 0.08)	0.43 (↓ 0.08)
DEVS-GEN (Ours)	**0.95 (↑ 0.50)**	**0.74 (↑ 0.44)**	**0.82 (↑ 0.16)**	**0.59 (↑ 0.16)**

6.4 Ablation Study: Scalability and Parallel Synthesis

A key architectural advantage of DEVS-GEN is its ability to decouple the synthesis process, which permits the parallel execution of structural planning and atomic model implementation. Predicated on the standard modeling practice where a coupled model aggregates at least two subcomponents (i.e., a branching factor $b \geq 2$), the depth of the system hierarchy grows logarithmically with the total number of components N. Consequently, the theoretical synthesis latency is governed by the critical path of this hierarchy rather than the sequential accumulation of all tasks. This structural property reduces the time complexity from linear $O(N)$ (characteristic of serial approaches), to logarithmic $O(\log N)$.

To validate this, we conducted an ablation study using GPT-5.2 on the DEVS-GEN framework, comparing the wall-clock time of serial execution versus parallel execution. Figure 4 reports the speed-up across the two stages.

Structural Planning Stage. Theoretically, the recursive planning algorithm allows independent branches of the system hierarchy to be decomposed concurrently. However, as observed in Figure 4, the empirical speedup is modest. This is primarily due to the scale of our current benchmark scenarios: the system hierarchies are relatively shallow, meaning the overhead of network requests and thread management outweighs the benefits of parallelizing the lightweight component classification tasks. We expect this speedup to become significant only in much larger, deeply nested systems.

Behavioral Synthesizing Stage. The acceleration is substantial, achieving a $\sim 4.7\times$ speedup. Since atomic models (leaf nodes) encapsulate the bulk of the complex logic (state transitions and logic implementation), their generation is the computational bottleneck. Our framework successfully isolates these tasks, allowing them to be synthesized simultaneously.

This result proves the scalability of DEVS-GEN. While standard agentic loops face super-linear growth in time and context costs as system complexity increases, our modular approach ensures that the synthesis time grows logarithmically, making it feasible for large-scale world modeling.

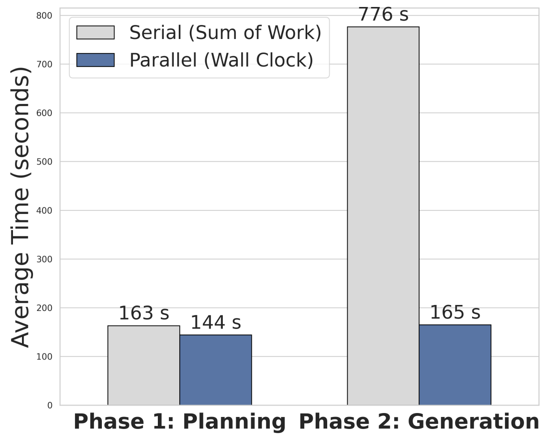

Figure 4: Ablation study on synthesis latency (using GPT-5.2). The chart compares the wall-clock time required for the Planning and Generation phases under serial and parallel execution modes. While the acceleration in the Planning phase is limited by the current benchmark scale, the Generation phase achieves a $4.7\times$ speedup, validating the scalability of the approach.

7 Conclusion

This paper introduces DEVS-GEN, a pipeline that synthesizes executable discrete-event world models from natural language. To systematically evaluate this paradigm, we construct a diverse benchmark of discrete-event scenarios, providing a rigorous trace-based evaluation framework. By leveraging the DEVS formalism to explicitly decouple structural routing from behavioral logic, our approach transforms unpredictable debugging sessions into highly parallelizable and reliable forward-pass construction. DEVS-GEN equips LLM agents with verifiable simulators, bypassing the compounding drift of implicit neural models and the inefficiency of iterative coding agents.

A comprehensive discussion of the framework's limitations (e.g., the scope of environment dynamics, model instruction adherence, and benchmark constraints) and future works is provided in App. H.

References

[1] David Ha and Jürgen Schmidhuber. Recurrent world models facilitate policy evolution. *Advances in neural information processing systems*, 31, 2018.

[2] Ignat Georgiev, Varun Giridhar, Nicklas Hansen, and Animesh Garg. Pwm: Policy learning with multi-task world models. In *The Thirteenth International Conference on Learning Representations*, 2025.

[3] Shibo Hao, Yi Gu, Haodi Ma, Joshua Hong, Zhen Wang, Daisy Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 8154–8173, 2023.

[4] Yu Gu, Kai Zhang, Yuting Ning, Boyuan Zheng, Boyu Gou, Tianci Xue, Cheng Chang, Sanjari Srivastava, Yanan Xie, Peng Qi, et al. Is your llm secretly a world model of the internet? model-based planning for web agents. *arXiv preprint arXiv:2411.06559*, 2024.

[5] Philip J. Ball, Jakob Bauer, Frank Belletti, Bethanie Brownfield, Ariel Ephrat, Shlomi Fruchter, Agrim Gupta, Kristian Holsheimer, Aleksander Holynski, Jiri Hron, Christos Kaplanis, Marjorie Limont, Matt McGill, Yanko Oliveira, Jack Parker-Holder, Frank Perbet, Guy Scully, Jeremy Shar, Stephen Spencer, Omer Tov, Ruben Villegas, Emma Wang, Jessica Yung, Cip Baetu, Jordi Berbel, David Bridson, Jake Bruce, Gavin Buttimore, Sarah Chakera, Bilva Chandra, Paul Collins, Alex Cullum, Bogdan Damoc, Vibha Dasagi, Maxime Gazeau, Charles Gbadamosi, Woohyun Han, Ed Hirst, Ashyana Kachra, Lucie Kerley, Kristian Kjems, Eva Knoepfel, Vika Koriakin, Jessica Lo, Cong Lu, Zeb Mehring, Alex Moufarek, Henna Nandwani, Valeria Oliveira, Fabio Pardo, Jane Park, Andrew

Pierson, Ben Poole, Helen Ran, Tim Salimans, Manuel Sanchez, Igor Saprykin, Amy Shen, Sailesh Sidhwani, Duncan Smith, Joe Stanton, Hamish Tomlinson, Dimple Vijaykumar, Luyu Wang, Piers Wingfield, Nat Wong, Keyang Xu, Christopher Yew, Nick Young, Vadim Zubov, Douglas Eck, Dumitru Erhan, Koray Kavukcuoglu, Demis Hassabis, Zoubin Gharamani, Raia Hadsell, Aäron van den Oord, Inbar Mosseri, Adrian Bolton, Satinder Singh, and Tim Rocktäschel. Genie 3: A new frontier for world models. 2025.

[6] Jichen Feng, Yifan Zhang, Chenggong Zhang, Yifu Lu, Shilong Liu, and Mengdi Wang. Web world models. *arXiv preprint arXiv:2512.23676*, 2025.

[7] McKinsey & Company. Unlocking the full power of automation in industrials. Technical report, McKinsey & Company, 2019. Accessed: 2026-05-05.

[8] Michael Chui, Mark Collins, Ankit Mittal, and Jonathan T. Warner. The Internet of Things: Catching up to an accelerating opportunity. Technical report, McKinsey Global Institute, 2021. Accessed: 2026-05-05.

[9] Grand View Research. Process mining software market size & share report, 2025–2030, 2025. Accessed: 2026-05-05.

[10] Yixia Li, Hongru Wang, Jiahao Qiu, Zhenfei Yin, Dongdong Zhang, Cheng Qian, Zeping Li, Pony Ma, Guanhua Chen, Heng Ji, et al. From word to world: Can large language models be implicit text-based world models? *arXiv preprint arXiv:2512.18832*, 2025.

[11] Bernard P Zeigler, Herbert Praehofer, and Tag Gon Kim. *Theory of modeling and simulation*. Academic press, 2000.

[12] José L. Risco-Martín, Saurabh Mittal, Kevin Henares, Román Cardenas, and Patricia Arroba. xDEVS: A toolkit for interoperable modeling and simulation of formal discrete event systems. *Software: Practice and Experience*, 53(3):748–789, March 2023.

[13] Dirk Bergemann, Soheil Ghili, Xinyang Hu, Chuanhao Li, and Zhuoran Yang. Training language models for bilateral trade with private information. *arXiv preprint arXiv:2604.16472*, 2026.

[14] Jiannan Xiang, Tianhua Tao, Yi Gu, Tianmin Shu, Zirui Wang, Zichao Yang, and Zhiting Hu. Language models meet world models: Embodied experiences enhance language models. *Advances in neural information processing systems*, 36:75392–75412, 2023.

[15] Hyungjoo Chae, Namyoung Kim, Kai Tzu-iunn Ong, Minju Gwak, Gwanwoo Song, Jihoon Kim, Sunghwan Kim, Dongha Lee, and Jinyoung Yeo. Web agents with world models: Learning and leveraging environment dynamics in web navigation. In *The Thirteenth International Conference on Learning Representations (ICLR 2025)*, 2025.

[16] Philipp Wu, Alejandro Escontrela, Danijar Hafner, Pieter Abbeel, and Ken Goldberg. Daydreamer: World models for physical robot learning. In *Conference on robot learning*, pages 2226–2240. PMLR, 2023.

[17] Mengkang Hu, Tianxing Chen, Yude Zou, Yuheng Lei, Qiguang Chen, Ming Li, Yao Mu, Hongyuan Zhang, Wenqi Shao, and Ping Luo. Text2world: Benchmarking large language models for symbolic world model generation, 2025.

[18] Renhao Zhang, Yilin Miao, and George Konidaris. Open-domain planning representations from texts. In *Proceedings of the Natural Language Reasoning and Structured Explanations Workshop (NLRSE)*. Association for Computational Linguistics, 2024.

[19] James Oswald, Kavitha Srinivas, Harsha Kokel, Junkyu Lee, Michael Katz, and Shirin Sohrabi. Large language models as planning domain generators. In *Proceedings of the Thirty-Fourth International Conference on Automated Planning and Scheduling*, pages 423–431, 2024. arXiv:2405.06650.

[20] Sadegh Mahdavi, Raquel Aoki, Keyi Tang, and Yanshuai Cao. Leveraging environment interaction for automated pddl translation and planning with large language models. In *Advances in Neural Information Processing Systems (NeurIPS 2024)*, 2024.

[21] Dongming Jin, Zhi Jin, Linyu Li, Zheng Fang, Jia Li, and Xiaohong Chen. A system model generation benchmark from natural language requirements, 2025.

[22] J. Chen, B. Hu, W. Diao, and Y. Huang. Automatic generation of sysml requirement models based on free-text requirements. In *Proceedings of the 2022 6th International Conference on Electronic Information Technology and Computer Engineering (EITCE 2022)*, pages 242–248, 2022.

[23] Han Zhou et al. Generating sysml behavior models via large language models. In *Proceedings of the 16th Asia-Pacific Symposium on Internetware (Internetware 2025)*, 2025.

[24] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael

Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code, 2021.

[25] Carlos E. Jimenez, John Yang, Alexander Wettig, Kilian Lieret, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024.

[26] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. arXiv:2306.03091.

[27] Jingzhe Ding, Shengda Long, Changxin Pu, Huan Zhou, Hongwan Gao, Xiang Gao, Chao He, Yue Hou, Fei Hu, Zhaojian Li, Weiran Shi, Zaiyuan Wang, Daoguang Zan, Chenchen Zhang, Xiaoxu Zhang, Qizhi Chen, Xianfu Cheng, Bo Deng, Qingshui Gu, Kai Hua, Juntao Lin, Pai Liu, Mingchen Li, Xuanguang Pan, Zifan Peng, Yujia Qin, Yong Shan, Zhewen Tan, Weihao Xie, Zihan Wang, Yishuo Yuan, Jiayu Zhang, Enduo Zhao, Yunfei Zhao, He Zhu, Liya Zhu, Chenyang Zou, Ming Ding, Jianpeng Jiao, Jiaheng Liu, Minghao Liu, Qian Liu, Chongyang Tao, Jian Yang, Tong Yang, Zhaoxiang Zhang, Xinjie Chen, Wenhao Huang, and Ge Zhang. Nl2repo-bench: Towards long-horizon repository generation evaluation of coding agents, 2025.

[28] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering, 2024.

[29] Wei Li, Xin Zhang, Zhongxin Guo, Shaoguang Mao, Wen Luo, Guangyue Peng, Yangyu Huang, Houfeng Wang, and Scarlett Li. Fea-bench: A benchmark for evaluating repository-level code generation for feature implementation, 2025.

[30] Jan Tretmans. Model based testing with labelled transition systems. In *Formal Methods and Testing*, volume 4949 of *Lecture Notes in Computer Science*, pages 1–38. Springer, 2008.

[31] Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.

[32] Gyeongmin Lee. An automated verification framework for devs-coupled models using devs-python. *Processes*, 13(5):1327, 2025.

[33] Ignacio Henares, David Ruiz-Martínez, and Eduardo Fernández-Medina. A unit testing platform to verify devs models. In *Proceedings of the 2020 Winter Simulation Conference (WSC)*, pages 2707–2718. IEEE, 2020.

[34] Jianhua Li and Hessam S. Sarjoughian. A testing framework for devs formalism implementations. In *Proceedings of the 2011 Winter Simulation Conference (WSC)*, pages 2735–2746. IEEE, 2011.

[35] Bernard P. Zeigler, James J. Nutaro, and Changhwan Seo. Combining devs and model checking: Concepts and tools for integrating simulation and analysis. *International Journal of Simulation and Process Modelling*, 12(1):2–15, 2017.

[36] Mohammad Gholami, James J. Nutaro, and Hessam S. Sarjoughian. Constrained-devs: A formalism for constraining discrete event system specifications. In *Proceedings of the 2017 Winter Simulation Conference (WSC)*, pages 1523–1534. IEEE, 2017.

[37] Ricardo da Silva, Alexandre de Melo, and Rogério de Lemos. On-the-fly verification of discrete event simulations by means of simulation purposes. In *Proceedings of the 2011 Symposium on Theory of Modeling & Simulation – DEVS Integrative M&S Symposium*, pages 73–80. Society for Modeling & Simulation International, 2011.

[38] Michael J. McLaughlin and Hessam S. Sarjoughian. Devs-scripting: A black-box test frame for devs models. In *Proceedings of the 2020 Winter Simulation Conference (WSC)*, pages 2666–2677. IEEE, 2020.

[39] Tsong Yueh Chen, S. C. Cheung, and S. M. Yiu. Metamorphic testing: A new approach for generating next test cases. *Technical Report HKUST-CS98-01*, 1998.

[40] Xingyao Wang, Simon Rosenberg, Juan Michelini, Calvin Smith, Hoang Tran, Engel Nyst, Rohit Malhotra, Xuhui Zhou, Valerie Chen, Robert Brennan, and Graham Neubig. The openhands software agent sdk: A composable and extensible foundation for production agents, 2025.

A Glossary of Key Terms

Term	Definition
CORE CONCEPTS	
World Model	The functional and conceptual predictive engine used by an autonomous agent to forecast the consequences of its actions and simulate environment dynamics over time. In this work, it specifically refers to models targeting discrete-event environments for long-horizon what-if reasoning.
Simulator ($\mathcal{M}$)	The concrete, executable software artifact (e.g., Python code) that operationalizes the world model. It acts as a black-box engine that accepts external interventions as inputs and emits observable event traces as outputs.
Event Trace ($\mathcal{T}$)	A chronological, structured sequence of discrete events emitted by the simulator during execution. Each record captures the timestamp, emitting entity, event type, and payload, serving as the observable behavior for specification-driven evaluation.
Environment Specification (`Spec`)	The natural-language description guiding the simulator generation, partitioned into operational configurations ($\texttt{Spec}_{ope}$) and behavioral descriptions ($\texttt{Spec}_{beh}$).
DEVS FORMALISM	
DEVS Model	The specific mathematical and structural representation of the environment constructed using the Discrete Event System Specification (DEVS) formalism. It serves as the underlying architecture for the executable simulator.
Atomic Model	A fundamental, indivisible DEVS component that maintains explicit internal state and dictates state-transition and timing logic (i.e., how the state evolves over time or in response to events). It contains no sub-components.
Coupled Model	A structural DEVS component that encapsulates multiple sub-components (which can be Atomic or other Coupled models). It contains no behavioral logic of its own, but defines the exact event routing (coupling) between the ports of its children and its own external interface.
Port	The explicit, typed communication interfaces through which DEVS components exchange discrete events, enforcing strict modular isolation.
PIPELINE ARTIFACTS	
`PlanTree`	The hierarchical intermediate data structure generated during the structural planning stage (Stage 1). It defines the entire system's architectural topology, capturing the component hierarchy and event routing graph.
`ModelPlan`	A strict interface contract and functional specification encapsulated within a specific node of the PlanTree. It dictates a component's model type, input/output port schemas, and expected behavioral logic, guiding the independent synthesis of that component in Stage 2.
`skeleton`	The initial, high-level blueprint inferred from the natural-language specification, outlining the basic module names, their parent-child relationships, and a brief description of their responsibilities before detailed routing is assigned.

B Formal Definition of Parallel DEVS

In this section, we provide the formal mathematical definitions of the Parallel DEVS (PDEVS) formalism [12] used to construct our discrete-event world models.

Atomic Models. An atomic model encapsulates the localized state and behavioral logic of an entity. We define an atomic model $\mathcal{M}_e$ for each entity $e \in \mathcal{E}$ as the 8-tuple:

$$\mathcal{M}_e = \left(X_e, Y_e, \mathcal{S}_e, \delta_{\text{int}}^e, \delta_{\text{ext}}^e, \delta_{\text{con}}^e, \lambda^e, t_a^e\right)$$

where:

- $X_e, Y_e \subseteq \Omega$ are the admissible input and output event alphabets.
- $\mathcal{S}_e$ is the local state space.
- $\delta^e_{\text{int}} : \mathcal{S}_e \to \mathcal{S}_e$ is the internal transition function.
- $\delta^e_{\text{ext}} : \mathcal{S}_e \times \mathbb{R}_{\geq 0} \times \mathcal{B}(X_e) \to \mathcal{S}_e$ is the external transition function. The real-valued argument denotes the elapsed simulation time since the last transition, and $\mathcal{B}(\cdot)$ denotes bags (multisets) of messages to handle concurrent inputs.
- $\delta^e_{\text{con}} : \mathcal{S}_e \times \mathcal{B}(X_e) \to \mathcal{S}_e$ is the confluent transition function, utilized to resolve conflicts when an external input arrives exactly at the scheduled internal transition time.
- $\lambda^e : \mathcal{S}_e \to \mathcal{B}(Y_e)$ is the output function.
- $t^e_a : \mathcal{S}_e \to \mathbb{R}_{\geq 0} \cup \{\infty\}$ is the time-advance function, determining the lifespan of the current state.

Operationally, an atomic model remains in state $s \in \mathcal{S}_e$ for $t^e_a(s)$ units of simulated time. Upon expiration, it emits outputs via λ^e and performs an internal transition via δ^e_{int}. If an input arrives before the scheduled internal transition, the model applies δ^e_{ext} using the elapsed time. If an input arrives exactly at the internal transition time, δ^e_{con} dictates the new state.

Coupled Models. Coupled models form the hierarchical structure of the system by composing sub-components and defining their routing topology. A coupled model $\mathcal{M}_c$ is defined as the 7-tuple:

$$\mathcal{M}_c = \left(X_c, Y_c, \mathcal{D}_c, \{\mathcal{M}_d\}_{d\in\mathcal{D}_c}, \text{EIC}_c, \text{EOC}_c, \text{IC}_c\right)$$

where:

- $X_c, Y_c \subset \Omega$ are the admissible input and output event alphabets.
- $\mathcal{D}_c$ is an index set of sub-components, where each $\mathcal{M}_d$ is either an atomic or a coupled model.
- $\text{EIC}_c \subset X_c \times \bigcup_{d\in\mathcal{D}_c} X_d$ is the External Input Coupling, which routes events from the coupled model's external interface to its sub-components.
- $\text{EOC}_c \subset \bigcup_{d\in\mathcal{D}_c} Y_d \times Y_c$ is the External Output Coupling, routing events from sub-components to the external interface.
- $\text{IC}_c \subset \bigcup_{d\in\mathcal{D}_c} Y_d \times \bigcup_{d\in\mathcal{D}_c} X_d$ is the Internal Coupling, which governs direct communication among sub-components.

This recursive structure yields a hierarchical representation whose root corresponds to the full simulator $\mathcal{M}$. Through this root, overall external inputs are injected to drive the execution of the environment.

C Structured Intermediate Representations

The `ModelPlanclass` serves as the central definition for any DEVS component. It strictly separates the functional logic from the logging requirements and enforces precise typing for input and output ports via `PortEntity` and `ProtocolSpec`.

```python
class TypedEntity(BaseModel):
    """
    Defines a specific argument, input port, or output port.
    """
    name: str = Field(..., description="Variable or port name. should be a valid Python identifier.")
    type: str = Field(
        ...,
        description="Python type hint (e.g., 'int', 'str', 'List[int]', 'Dict[str, float]'). Keep structures simple."
    )
    structure: str = Field(
        ...,
        description="Structure of the data. CRITICAL: If 'type' is a complex structure (like dict or list), "
                    "you MUST detail the expected format, keys, and value constraints here."
    )

```

```
class PortEntity(TypedEntity):
    """
    Defines a specific argument, input port, or output port.
    """
    protocol: ProtocolSpec = Field(
        ...,
        description="The protocol for this port. Including initiation and data exchange."
    )

class ModelPlan(BaseModel):
    """
    Detailed functional specification and Interface definition.
    Used by both Atomic and Coupled models to define their EXTERNAL behavior and ports.
    """
    function: str = Field(..., description="The Responsibility & Workflow & Logic, as well as user specified details.")
    logging: str = Field(..., description="logging requirements. e.g. What specific data should be logged for debugging/analysis.")

    model_init_args: list[TypedEntity] = Field(
        default_factory=list,
        description="Parameters required to initialize the model class (e.g., initial_count, processing_time)."
    )
    input_ports: list[PortEntity] = Field(
        default_factory=list,
        description="Data inputs received by this model."
    )
    output_ports: list[PortEntity] = Field(
        default_factory=list,
        description="Data outputs sent by this model."
    )
```

Listing 1: Pydantic definitions for `ModelPlan`

D Agent Prompt Templates

This section presents the core prompt templates used in our generation pipeline. Each prompt is designed using a "Role-Task-Constraint" structure, injecting the structured context data defined in Appendix C.

D.1 Structural Planning Stage Prompts

The **Infer Global Skeleton Prompt** is responsible for generating the general skeleton.

Infer Global Skeleton Prompt Template

```
## [Role]
You are a **DEVS System Architect**. Your task is to design the overall module hierarchy for a DEVS simulation system.

## [Input]
**System Name**: `{root_name}`
**Requirements**:
{requirements}

## [Task]
Decompose the system into a hierarchical module structure. Return a list of modules.

## [Rules]
```

```
1. The first module MUST be the root system: `{root_name}`. It should be a coupled model with children.
2. Every name mentioned in any `children_names` MUST appear as a `name` somewhere in the list.
3. Use hierarchical decomposition: group related functionality into intermediate coupled models before reaching atomic models.
4. Keep descriptions SHORT (1-2 sentences). Only state what the module does.
5. Module names should be valid Python class names (PascalCase, no spaces/special chars).
6. Atomic (leaf) modules should have `children_names: []`.
7. Do NOT over-decompose. Less than 4 levels of hierarchy is usually sufficient.
8. Each coupled model should have at least 2 children (you can claim one type being instantiated multiple times)
9. DEVS Principles:
  - A system is hierarchical.
  - Atomic models have behavior (state machines) but NO sub-components.
  - Coupled models have sub-components and routing (couplings) but NO behavior.
  - The input/output ports of a model can only connect to a sibling (IC) or be a proxy of parent model (EIC, EOC).
  - Define the data_flows to establish the exact communication topology between the modules you create.

## [IMPORTANT: KEEP IT SIMPLE]
- **Minimize the number of modules**: Only create modules that are truly necessary.
- **Prefer shallow hierarchies**: A flat structure with fewer levels is better than a deep one.
- **Each module should have a clear, distinct responsibility**: Avoid overlapping functions.
- **Atomic modules should be self-contained**: Each should implement a complete, coherent piece of functionality.

## [Field Guidance]
- `name`: Valid Python identifier, PascalCase. Example: "InputHandler", "CoreProcessor".
- `description`: 1-2 sentences stating what the module does. Example: "Validates and normalizes incoming data.". Also describe the data flow inside the model (EIC, EOC, IC).
- `children_names`: List of child module names. Empty list `[]` for atomic (leaf) modules.

## [Example]
For a system "DataPipeline" with requirements "ingest, transform, and output data":
- modules:
  - name: "DataPipeline", description: "Top-level coordinator. Routes data through ingestion, transformation, and output stages.", children_names: ["Ingester", "Transformer", "Emitter"]
  - name: "Ingester", description: "Reads raw data from source and validates format.", children_names: []
  - name: "Transformer", description: "Applies transformation rules to validate data.", children_names: ["Mapper", "Filter"]
  - name: "Mapper", description: "Maps input fields to target schema.", children_names: []
  - name: "Filter", description: "Removes invalid or duplicate records.", children_names: []
  - name: "Emitter", description: "Writes transformed data to target destination.", children_names: []
```

Following the global skeleton inference, the architecture refinement process uses two distinct prompt templates depending on the model type being processed. This corresponds to the top-down traversal defined in Stage 1.

The **Refine Coupled Plan Prompt** serves a dual purpose: it defines the internal routing (EIC, IC, EOC) for the current coupled model, and simultaneously generates the preliminary plan for all of its direct children.

Refine Coupled Plan Prompt Template

```
## [Role]
You are a **DEVS System Architect**.

## [Input]
**Target Module**: `{target_name}`
**Children in Global Plan**: `{children_names_str}`
**Original Requirements**:
{requirements}
```

```

**Global Plan Overview** (full module hierarchy):
{global_plan_str}

**Model's Simple Plan** (this model's initial interface from parent):
{parent_simple_str}
**Parent's Detailed Plan** (system context):
{parent_detail_str}

## [Task]
Generate a detailed specification for the COUPLED module `{target_name}` and simple specifications for its direct children ({children_names_str}).

### STEP 1: Design Coupled Wrapper (`detailed_plan`)
- **function**: Describe the overall purpose and capability of this entire subsystem as a unified whole. While the coupled wrapper itself only contains structural connections (no active routing/state logic), this field should summarize what the encapsulated subsystem achieves.
- **model_init_args**: inherit from Parent's Simple Plan.
- **input_ports / output_ports**: inherit from Parent's Simple Plan.

### STEP 2: Design Children Contracts (`children_plans`)
- **function**: 1-2 sentences describing child's responsibility.
- **logging**: 1-2 sentences, mention what logging items should be covered.
- **model_init_args**: full args (name/type/structure); always start with name (str) and parent (Coupled | None).
- **input_ports / output_ports**: full port definitions for sibling and parent matching. MUST make sure the structures of ports match strictly.

### STEP 3: Design `coupling_specification`:
- MUST define the network routing here using ONLY these 3 strict DEVS patterns. List them clearly line-by-line:
    1. **EIC (External Input Coupling)**: `parent.IN.port_name -> child.IN.port_name`
    2. **IC (Internal Coupling)**: `child_A.OUT.port_name -> child_B.IN.port_name`
    3. **EOC (External Output Coupling)**: `child.OUT.port_name -> parent.OUT.port_name`
- **CRITICAL COUPLING RULES**:
    1. **NO HALLUCINATIONS**: Every port name you use MUST EXACTLY MATCH either the Parent's inherited ports or the Children's ports you defined in STEP 2.
    2. Not all ports need parent connections; children can communicate entirely via IC.

## [Field Guidance]
- For ANY `dict` or `list` types, you MUST use strict Python representation (e.g., `{'sequence_number': int, 'is_retry': bool}`).
- Port protocol: Must include initial_state (state at T=0), initial_signal (signal at startup), description.
```

The **Refine Atomic Plan Prompt** takes the preliminary simple plan assigned by its parent coupled model and expands it into a detailed plan, focusing strictly on state transitions, timing, and local logic without altering the assigned interfaces.

Refine Atomic Plan Prompt Template

```
## [Role]
You are a **DEVS System Architect**.

## [Input]
**Target Module**: `{target_name}`
**Children in Global Plan**: None (leaf)
**Original Requirements**:
{requirements}

```

```
**Global Plan Overview** (full module hierarchy):
{global_plan_str}

**Model's Simple Plan** (this model's exact contract assigned by parent):
{parent_simple_str}
**Parent's Detailed Plan** (system context):
{parent_detail_str}

## [Task]
Generate a detailed specification for the ATOMIC module `{target_name}`.

### For the detailed ATOMIC `{target_name}` plan:
- **function**: Describe pure responsibility, state machine, event handling, and timing. Do
    NOT describe logging here.
- **logging**: Extract ALL logging/output requirements from the original requirements that
    apply to this model. Include payload structure, format, and timing.
- **model_init_args**: Strictly copy from Model's Simple Plan.
- **input_ports / output_ports**: Strictly copy from Model's Simple Plan. DO NOT invent new
    ports.

## [Field Guidance]
- For ANY `dict` or `list` types, you MUST use strict Python representation (e.g.,
    `[{'job_id': int, 'priority': float}]`).
- Port protocol: Must include initial_state (state at T=0), initial_signal (signal at
    startup), description.
```

D.2 Behavioral Synthesizing Stage Prompts

The **Model Creator** synthesizes the actual executable Python code. To ensure high-quality generation, we inject a set of global engineering standards alongside model-specific instructions (Atomic vs. Coupled).

The main prompt template is:

Main Code Generation Prompt Template

```
## [Task]
Construct a complete Python file containing a **{model_type} DEVS model** named `{name}` using
    `xdevs.py`.

{global_standards}

{model_specific_instructions}

{feedback}

## [Context Info]
**Sub-Models (for Coupled definitions)**:
{sub_models}

**System Context**:
(The environment around this model)
(You must especially guarantee the JSONL output requirements are met.)
{context_str}

## [Utils]
{util_desc}

## [Class Definitions]
{definitions}

## [Specification]
```

```
The ports, logic, logging dict keys, and parameters of the model should strictly follow the
    specification (including their types, functions), only two can be added / modified: in
    __init__ args, `name: str`, and `parent: Coupled | None`:
{spec}

## [Reference Example]
Refer to this example for coding style and imports:
{example}

## [Output]
Return the Python code enclosed in <python_code> tags.
Do not use markdown backticks.

Example:
Think step by step, decompose the requirements and state machine.
Finally the enclosed code.
<python_code>
class MyModel(Atomic):
    ...
</python_code>
```

Global Engineering Standards (Injected into Main Prompt)

```
## [Global Standards]
Adhere to the following engineering standards for all model types:

### 1. Imports \& Dependencies
- **Whitelist**: Restrict imports to the following packages: `numpy`, `math`, `random`,
    `time`, `pandas`, `xdevs` (and `xdevs.models`).
- **Project Utils**: Import necessary utilities (e.g., `get_sim_logger`, `get_current_time`)
    from `devs_project.devs_utils.xxx`. Refer to [Utils] for detailed import statements.
- Other submodels in the project can be imported as needed.

### 2. Data Protocol \& Typing
- You are only allowed to use the following Atomic Primitives and Composite Types in ports and
    arguments.
- **Atomic Primitives**: Only `int`, `float`, `str`, `bool` instances for all base values.
- **Composite Types**: Only `dict` and `list`.
- **Consistency**: Ensure that all ports and arguments are the same as those stated in the
    [Specification].
- **Recursive Schema Definition**:
    - Recursively define the structure of all composite arguments (in `__init__`) and port
    data types (in Docstrings).
    - Continue the definition until all fields resolve to Atomic Primitives.
    - **Dictionaries**: Explicitly list every Key name, the Type of its Value, and the
    explanation of it.
    - **Lists**: Explicitly state the Type of elements contained in the list.
- Special: only the `parent` argument of the `__init__` method can be of type `Coupled | None`.

### 3. Type Docstring Schema
- **Structure**: Follow this hierarchy for describing types in Class and Method docstrings:
    - Root: `name (type): Description`
    - Dict Keys: Indent 2-4 spaces, `key_name (type): Description`
    - List Items: Indent 2-4 spaces, `- (type): Description`
- *Exception*: For Logging, you can refer to the structure described in the ports section, as
    they are often the same.
- **Example**:
    ```python
    \"\"\"
    ...
    - in_packet (dict): Network packet.
        header (dict): Protocol header.
```
```

````
            src_ip (str): Source IP.
        payload (str): Data content.
    - batch_updates (list[dict]): Updates.
        - (dict): Single update.
            node_id (int): Target ID.
    \"\"\"
    ```

### 4. Coding Conventions
- **Explicit Configuration**: Define all configuration parameters explicitly in `__init__`. Omit `*args` and `**kwargs`.
- **Logging**: Log all key events using `self.logger.info({{"key": "value"}}, log_type=...)`. The main msg must be a dict. They must have the exact precise structure and keys as the context (especially the system goal) and the Model Specification. You should explain the structure of the data in the docstring. However, you only need to list the logging happen in this model, those in submodels are strictly forbiddened.
- **Parameter Storage**: Store internal hardcoded parameters (not passed via `__init__`) in a `self.param` dictionary.
````

Atomic Model Instructions (Injected into Main Prompt)

````
### [Atomic Model Specifics]
0. Must import: `from xdevs.models import Atomic, Coupled, Port`. Atomic for inherit, Coupled for __init__ arg type.
1. **Inheritance**: Inherit from `Atomic`.
2. **Docstring**: The class MUST include a standard class docstring strictly following this format:
    ```python
    class {name}(Atomic):
        \"\"\"
        Function:
            - ...General function
            - ...Every state: how it transfer, and what to output after the state is over.
        Logging in this model:
            - ...
            - ...
        Input Ports:
          - port_name (type): description
            structure: ...
            protocol: initialize: ... ; process: ...
        Output Ports:
          - port_name (type): description
            structure: ...
            protocol: initialize: ... ; process: ...
        \"\"\"
    ```
3. **Constructor (`__init__`)**:
    - Signature: `def __init__(self, name: str, parent: Coupled | None, <explicit_config_args>)`
    - Docstring: should have a docstring describing the arguments, including the detailed type and description. using the following format:
        ```python
        \"\"\"
        Args:
            name (str): The unique name of the model.
            parent (Coupled | None): the parent model. If None, the model is a root model.
            arg_name1 (type): description
        \"\"\"
        ```
    - Steps:
        1. Call `super().__init__(name)`.
        2. Assign `self.parent = parent`.
````
````

```
        3. Initialize logger: `self.logger = get_sim_logger(self)`.
        4. Register Ports: Use `self.add_in_port(Port(type, "name"))` and
    `self.add_out_port(Port(type, "name"))`.
        5. Initialize State: Set member variables and call `self.hold_in(phase, time)`.
        6. Log creation: `self.logger.info({{keys: values, ...}}, log_type=...)`
4. **Core Behaviors**:
    - Implement `initialize(self)`: Set initial state. Set phase/sigma using
    `self.hold_in(phase, time)`. Log initialization.
        - It can not send any output. If you need to send a initial signal (e.g. report you
    are ready), you can use `self.hold_in(phase, time)` to schedule the event, prepare the
    payload, and send it in `lambdaf`.
        - If any port has `initial_signal` to emit immediately or after a duration, the
    `initialize` method **MUST** schedule an output event using `self.hold_in("SOME_STATE",
    duration)`.
    - Implement `lambdaf(self)`: Only do the output, any other operations should be done in
    the following `deltint`:
        - Send output via `self.output["port"].add(payload)`.
        - DO NOT change the state, sigma, kpi_counter, etc. Leave that to the following
    `deltint`.
    - Implement `deltint(self)`: Only do the following:
        - Get the old internal state: `self.phase`. Get total time(from last state change to
    expected next state change, which is just the sigma set last time): `self.ta()`.
        - Handle internal timeouts. And update the internal queue / kpi_counter / etc.
    accordingly. (Because deltint is called right after lambdaf, which means the output of the
    old state is already sent).
        - Prepare the payload of the output of the next phase. Make sure the prepared payload
    is the one used in `lambdaf` of the new phase.
        - Always schedule next internal event in the end: `self.hold_in(phase, sigma)`. If not
    interrupted, the model will emit the output prepared after sigma time units.
        - Log events (if needed).
    - Implement `deltext(self, e)`: Only do the following:
        - Handle external events (`self.input["port"].values`).
        - Get internal state: `self.phase`. Get total time(from last state change to expected
    next state change, which is just the sigma set last time): `self.ta()`.
        - Prepare the payload of the next lambdaf. Make sure the prepared payload variable is
    the one used in `lambdaf`.
        - Always schedule next internal event in the end: `self.hold_in(phase, sigma)`.
        - Log events (if needed).
    - Implement `exit(self)`: Cleanup and final stats logging.
    - **Event Handling Logic**:
        - **Execution Sequence (CRITICAL)**: `lambdaf` will send outputs before `deltint`
    schedules the next internal event. Thus, the payload sent in `lambdaf` should be prepared
    in the previous `deltint`, `deltext`, or `initialize`.
        - **Confluent Events (`deltcon`)**: By default, internal events (`deltint`) take
    precedence over external events when they occur simultaneously. Explicitly override the
    `deltcon(self)` method ONLY IF you need to change this logic (e.g., to process external
    events first).
        - **Initialization**: Realize the ports.protocol's initialize descriptions:
            - initial_signal: If a signal or information should be sent at initialization(i.e.
    protocol.initial_signal), you can use `self.hold_in("INIT", 0)` to schedule the event and
    send it in `lambdaf`. This is the only way to send a signal at initialization.
            - initial_state: modify the logic and initial values to make sure it is realized.
5. **logging requirements**: make sure all the events required are logged. And the keys and
    structures of the logs must match the Specification exactly.
```

Coupled Model Instructions (Injected into Main Prompt)

```
### [Coupled Model Specifics]
0. Must import: `from xdevs.models import Atomic, Coupled, Port`
1. **Inheritance**: Inherit from `xdevs.models.Coupled`.
2. **Docstring**: The class MUST include a standard class docstring strictly following this
    format:
```

```
    ```python
    class {name}(Coupled):
        \"\"\"
        Function:
          - ...
          - ...
          - Sub-models:
            - sub_model_class_name: name=sub_model_instance_name. Brief description.
        Logging in this model:
          - ...
          - ...
        Input Ports:
          - port_name (type): description
            structure: ...
            protocol: initialize: ... ; process: ...
        Output Ports:
          - port_name (type): description
            structure: ...
            protocol: initialize: ... ; process: ...
        \"\"\"
    ```
3. **Container Logic**: Treat this class as a pure structure container. Implement ONLY `__init__`.
4. **Sub-models Imports**: Use relative imports for sub-models (e.g., `from .folder.file import SubModelName`).
5. **Constructor (`__init__`)**:
    - Signature: `def __init__(self, name: str, parent: Coupled | None, <explicit_config_args>)`
    - Docstring: should have a docstring describing the arguments, including the detailed type and description. using the following format:
        ```python
        \"\"\"
        Args:
            name (str): The unique name of the model.
            parent (Coupled | None): the parent model. If None, the model is a root model.
            arg_name1 (type): description
        \"\"\"
        ```
    - Steps:
        1. Call `super().__init__(name)`.
        2. Assign `self.parent = parent`.
        3. Initialize logger: `self.logger = get_sim_logger(self)`.
        4. Register Ports: Use `self.add_in_port(...)` and `self.add_out_port(...)`.
        5. Instantiate Components: Create sub-model instances and register them via `self.add_component(instance)`.
        6. Define Couplings: Use `self.add_coupling(src, dst)` for:
            - **EIC**: `self.input["port_name"]` -> `sub.input["port_name"]`
            - **IC**: `sub_a.output["port_name"]` -> `sub_b.input["port_name"]`
            - **EOC**: `sub.output["port_name"]` -> `self.output["port_name"]`
        7. Log creation: `self.logger.info(...)`
    - Note: For steps 5-6, you should refer to Sub-Models to get the right init args names and port names. These information can be used as a correction and supplement to the coupling logic (in case some names are inconsistent).
```

The **Model Summarizer** implements the bottom-up adaptation mechanism. It analyzes the generated code to extract the "Ground Truth" interface, which is then used to guide the generation of the parent coupled model.

Summarizer Prompt Template

```
## [Task]
Analyze the provided Python code for a DEVS model.
Extract the model's metadata into a strict structure based on the schema below.
```

```
## [Rules]
- **model_init_args**: You must extract all the arguements from the `__init__` method (except
    `self`). You should make sure that the usage of all args are clear (especially for the
    str, dict types, and possible options).
- **logging**: Look for the docstring and logging usage to fill `logging`. You must clearly
    state the `log_type` and the structure of main msg.
  - You only need to extract the logger used in this model. The logging inside sub-models are
    not needed, as they are handled by their own models.
  - If it is described in the docstring, just copy the releted part in docstring, you can add
    more details if needed. (e.g. to keep the name of data clear, or other needs. )
- **ports**: Refer to the docstring for port definitions.
- **function**: Look for the main logic of the model. For communication protocols, find how it
    starts, send, receive, and ends(if any).
  - For coupled models, you must copy the submodels described in the docstring, especially the
    relation between the class name and the instance name.

{feedback}

## [Sub-Models Info]
{sub_models}

## [Code]
```python
{code}
```
```

D.3 Godot Generation Agent Prompt

We use the following prompt for the Godot construction.

Godot Generation Agent Prompt Template

```
You are a coding agent using godogen to build one runnable Godot scene for a DEVS simulator.

Target engine:

- Godot 4.6 (STRICT). All generated code and APIs must be compatible with Godot 4.6 only.

Available inputs:
1) DEVS_examples (reference behaviors/patterns),
2) one scenario YAML file,
3) one DEVS simulator package (if provided in workspace),
4) image generation API config below.

Primary goal:
Produce a Godot project/scene that can be opened and run directly in Godot 4.6, and that
    visualizes the DEVS scenario behavior correctly.

Hard requirements:
1) Build scene logic from YAML + DEVS_examples (+ simulator package if available).
2) Connect runtime events via MQTT (or provided runtime stream) and update scene
    deterministically.
3) Generate and APPLY textures for scene elements (do not leave plain placeholders):

   - background image
   - source/facility visual
   - destination visual
   - moving entities (e.g., aircraft/vehicles/agents)
   - cargo/package/queue-related visuals
   - core UI icons for status panels
     4) Ensure all major visual elements are textured in final scene.
```

```
    5) Prioritize delivering runnable output over long explanations.

Layout and visibility requirements:
6) Scene must be fully visible and readable at:

   - 1152x648
   - 1366x768
   - 1920x1080
    7) Use responsive UI anchors and scalable center viewport.
    8) Avoid clipping critical panels/text and avoid overlap between key UI and moving
    entities.

Runtime motion requirements:
9) Scene must show visible motion after launch.
10) If live MQTT/runtime stream is available, drive updates from incoming events.
11) If live stream is unavailable within 3 seconds, switch to replay/demo mode (same event
    schema) so the scene is still animated and demonstrable.

Godot 4.6 compatibility rules (STRICT):
12) Do not use Godot 3-only APIs.
13) For UDP in Godot 4.6:

   - use PacketPeerUDP.new()
   - bind to local port
   - poll packets in _process(delta) using get_available_packet_count()
   - read via get_packet()
   - parse payload and apply updates
    14) NEVER call nonexistent methods such as:
   - PacketPeerUDP.set_blocking_mode(...)
    15) On bind/read/parse failure: log error and continue safely without crashing.

GDScript type-safety rules (STRICT):
16) Assume warnings may be treated as errors. Generated code must be warning-free.
17) Do NOT rely on inferred typing when RHS may be Variant.
18) For values from JSON/Dictionary/Array/get(), always use explicit type annotations plus
    cast/conversion.
19) Avoid := when RHS can be Variant; prefer explicit declarations.
20) Always convert Dictionary.get(...) values before assignment/use.
21) Before final output, fix all type warnings.

Type-safe style examples:

- var msg: Dictionary = JSON.parse_string(text) as Dictionary
- var payload: Dictionary = msg.get("payload", {}) as Dictionary
- var event_type: String = str(msg.get("event_type", ""))
- var ts: float = float(msg.get("timestamp", 0.0))
- var records: Array = msg.get("records", []) as Array

Execution policy:

- Do not spend time on extended self-check reports.
- Do not iterate endlessly on optimization.
- Finish when scene is runnable, textured, visibly animated, and warning-free.
- If one asset generation fails, retry briefly, then generate a substitute asset and continue.

Final output:
A) Runnable Godot scene/project files (Godot 4.6 compatible)
B) Brief run note: which scene to open and run
C) Brief list of generated texture files actually used in scene
D) Confirmation that no warnings remain (especially Variant-inference warnings)
```

E Benchmark Scenario Specifications

This section provides detailed descriptions for the 7 benchmark scenarios summarized in Table 1. For each scenario, we describe the system objective, the constituent entities, and the governing dynamics. We abopted this originally from `https://github.com/SimulationEverywhere/Cadmium-Simulation-Environment`, a widely acknowledged repository consisting various simulation tasks designed and implemented by human experts.

E.1 Online Banking System (IOBS)

Domain: Service Operations **Dynamics Type:** Pipeline with Probabilistic Routing

- **Overview:** This scenario models a linear transaction processing pipeline for an online banking system. It simulate the sequential validation of user requests through multiple security and processing layers.
- **Entities:** The system consists of a chain of five modules: Account Access Manager, Account Number Verifier, Password Verifier, Bill Payment Manager, and Transaction Process Manager.
- **Dynamics:** The system operates as a single-direction pipeline. A request enters at given time and passes through each stage with a fixed processing delay (10s).
  - **Probabilistic Branching:** At the ANV and PV stages, requests have a 50% probability of failing verification. Failed requests are terminated immediately (dropped from the pipeline), while valid requests proceed.
  - **State Updates:** The final TPM stage maintains a persistent state (account balance) that is updated only upon the successful completion of the entire pipeline chain.

E.2 O-Train Light Rail (OTrain)

Domain: Transportation **Dynamics Type:** Schedule-driven with Serial Delays

- **Overview:** This scenario simulates a light rail transit system with a single train shuttling bi-directionally between 5 stations (Bayview to Greenboro). It models the interaction between the train schedule and stochastic passenger arrival.
- **Entities:** *Train* (mobile resource), *Stations* (fixed nodes), and *Passengers* (transient entities).
- **Dynamics:**
  - **Schedule-Driven Movement:** The train adheres to a strict timetable, moving between stations with fixed travel intervals.
  - **Serial Serialization:** Unlike simple fluid queues, boarding and alighting are modeled as serial processes. Each passenger takes 0.025s to board or alight, creating a delay that is linearly proportional to the number of passengers.
  - **Stochastic Generation:** Passengers are generated at stations using a randomized interval (Normal distribution), with specific origin-destination constraints.

E.3 Epidemic Compartmental Model (SEIRD)

Domain: Biological Systems **Dynamics Type:** Continuous (ODE)

- **Overview:** This scenario implements a classic SEIRD (Susceptible-Exposed-Infective-Recovered-Deceased) model to simulate disease spread within a closed population.
- **Entities:** The population is treated as an aggregate divided into five compartments (S, E, I, R, D).
- **Dynamics:** Unlike other event-driven scenarios, this model approximates continuous system dynamics using numerical integration (Forward Euler method).
  - **State Evolution:** State updates occur at fixed time steps (days). The transitions between compartments are governed by differential equations based on parameters such as transmission rate (), incubation period, and mortality rate.
  - **Conservation:** The model must strictly enforce population conservation at every time step despite floating-point operations.

E.4 Offline File Transfer (OfflineFileTransfer)

Domain: Network Systems **Dynamics Type:** Dual-Loop FSM with Interrupts

- **Overview:** This scenario simulates a "Dropbox-like" synchronization system where a file upload process and a download process are decoupled by an intermediate server buffer.
- **Entities:** *Sender*, *Server* (acting as a buffer), *Receiver*, and four reliable *Subnets* (channels).
- **Dynamics:** The system features two asynchronous loops:
  - **Dual-Loop Coupling:** The Upload Loop (Sender Server) and Download Loop (Server Receiver) operate independently but are coupled via the Server's storage queue. Both loops utilize the Alternating Bit Protocol (ABP) for reliability.
  - **External Interrupts:** The simulation is driven by a command stream. The request command acts as an external interrupt (or valve) that can dynamically pause or resume the download loop, requiring the system to handle state persistence during interruptions.

E.5 C.5. Alternating Bit Protocol (ABP)

Domain: Network Protocols **Dynamics Type:** Stop-and-Wait with Deterministic Noise

- **Overview:** A standard implementation of the Alternating Bit Protocol (ABP) to achieve reliable data transmission over an unreliable channel.
- **Entities:** *Sender*, *Receiver*, and two *Subnets* (Forward and Backward channels).
- **Dynamics:**
  - **State Machine:** The Sender implements a "Stop-and-Wait" FSM. It sends a packet and blocks until a valid ACK (with the matching bit) is received or a timeout occurs.
  - **Deterministic Noise:** Packet loss is not random but governed by a Linear Congruential Generator (LCG) inside the subnets. This ensures the simulation is strictly reproducible: packet fate (drop vs. pass) is pre-determined by the seed, testing the model's ability to implement precise algorithmic logic.

E.6 C.6. Strategic Airlift Operations (StrategicAirlift)

Domain: Logistics **Dynamics Type:** Active Reneging & Resource Cycling

- **Overview:** A complex logistics simulation managing the transport of cargo pallets via a fleet of aircraft.
- **Entities:** *Facility* (Source), *Loading Queue*, *Fleet Coordinator*, multiple *Aircraft*, and *Destination*.
- **Dynamics:**
  - **Active Reneging:** Pallets in the queue have a finite lifespan. The queue must actively monitor expiration times and autonomously discard (renege) items that stay too long, even without external events.
  - **Resource Scheduling:** Aircraft follow a multi-stage state cycl. The Coordinator must match available cargo with idle aircraft, handling resource contention.

E.7 C.7. Barbershop (Barbershop)

Domain: Service Queueing **Dynamics Type:** Blocking & Handshake Signaling

- **Overview:** A classic queuing simulation of a barbershop with a reception area and a two-stage service process (Inspection and Cutting).
- **Entities:** *Reception Desk*, *Inspection Station*, and *Cutting Station*.
- **Dynamics:**
  - **Finite Blocking:** The reception queue has a hard capacity (limit 8). Arrivals when the queue is full are strictly rejected (Blocking).
  - **Inter-module Signaling:** The two service stages (Inspection and Cutting) operate sequentially. The Inspection module must wait for a "done" signal from the Cutting module (Handshake) before it can release the current customer and accept a new one from Reception.

F Case Study Artifacts

To illustrate the stepwise transformations of our generation pipeline, we present the concrete artifacts generated for the Alternating Bit Protocol (ABP) case study. This section follows the chronological execution of the pipeline, demonstrating how unstructured natural language is progressively refined into a rigorous, executable DEVS model.

F.1 Input Specifications

The pipeline begins with the user-provided specifications. Listing **??** defines the operational logic and constraints in natural language, while Listing **??** dictates the simulation's interface, I/O formats, and configuration parameters. Notably, these inputs contain no explicit DEVS architectural blueprints.

```
### Scenario: Reliable Data Transfer with Deterministic Noise Interference
1. System Objective: Design a communication system consisting of a Sender, a Receiver, and two uni-directional transmission channels (Subnets). The goal is to transmit a sequence of packets reliably using an Alternating Bit Protocol (ABP) despite deterministic packet loss in the channels.

2. Entity Behaviors:
The Sender:
    - Accepts a single control input at the start of simulation: the total number of packets to send.
    - Before sending each packet, the Sender must undergo a preparation delay (default 10ms, configurable via --sender_delay).
    - The Receiver must maintain a buffer with capacity 1. During the busy period, it must buffer the only received packet, and process the packet immediately after the busy processing delay. When multiple packets arrive, only the first one is stored.
    - Sends packets sequentially through Subnet1. Each packet contains a sequence number (1, 2, ...) and a control bit (alternating between 0 and 1). The first bit is 0.
    - After sending a packet, it starts a timer (default 20ms, configurable via --timeout).
    - Stop-and-Wait Logic: It must not send the next packet until it receives a correct Acknowledgment (ACK) for the current one.
    - Retransmission: If the timer expires before a valid ACK is received, the Sender retransmits the same packet and restarts the timer.
    - Validation: An ACK is valid only if its bit matches the control bit of the current packet.
    - After sending the specified total number of packets, the Sender stops automatically.

The Receiver:
    - Upon receiving a packet, it undergoes a processing delay (default 10ms, configurable via --receiver_delay) before processing.
    - The Receiver must maintain a buffer with capacity 1. During the busy period, it must buffer the only received packet, and process the packet immediately after the busy processing delay. When multiple packets arrive, only the first one is stored.
    - After the processing delay, it extracts the control bit and immediately sends back an ACK packet containing that same bit through Subnet2.

The Subnets (Channels):
    - There are two independent channels: Subnet1 (Sender -> Receiver) and Subnet2 (Receiver -> Sender).
    - Latency: Every packet takes exactly 3ms (configurable via --channel_delay) to traverse.
    - Deterministic Noise & Loss Model: Each subnet independently simulates interference using a deterministic formula.
        - Each Subnet maintains an internal "noise level" value x, initialized to exactly the seed value (provided via --seed).
        - Packet Fate Determination: When a packet arrives at a subnet, calculate a new noise level: x_new = (17 * x_old + 11) mod 100.
```

```
        - If x_new < 10, the interference is too high, and the packet is dropped (vanishes). Otherwise, it is transmitted normally after the channel delay.
        - After determination, update the noise level for the next packet: x_old = x_new.
        - Timing: The noise calculation and fate determination happen immediately when the packet arrives at the subnet.

3. Scenario Constraints:
    - Time Unit Mapping: 1.0 simulation time unit = 1 Millisecond (ms).
    - System starts at time 0.0 with all components initialized to idle states.
```

Listing 2: Behavioral Specification

```
1. Command Line Arguments:
The script must accept the following named arguments:
* `--total_packets` (int): The total number of packets the Sender intends to send in one session triggered by a START_BATCH command.
* `--seed` (int): The initialization seed for the noise generator of both sides (the `x` value in the LCG formula). Default: 42.
* `--timeout` (int): Sender's timeout duration in ms. Default: 20.
* `--sender_delay` (int): Sender preparation delay in ms. Default: 10.
* `--receiver_delay` (int): Receiver processing delay in ms. Default: 10.
* `--channel_delay` (int): Subnet transmission delay in ms. Default: 3.
* `--simulate_time` (int): The total simulation time to run in ms. Default: 1000.

2. stdin Format:
* No stdin input is required for this simulation. The system uses command line arguments for configuration.

3. **Standard Output (stdout)**:
* Format: JSONL, one independent JSON object per line
* Each record MUST follow the format: `{"time": <float>, "entity": <str>, "event": <str>, "payload": <dict>}`
* **Event Types and Formats**:
    Sender Events:
    - event: `delay_start` (Sender starts preparation delay)
    - time: Sender's current time
    - entity: `"sender"`
    - payload: `{"type": "preparation", "duration": <float>}`
    - Trigger: When Sender starts preparing a packet

    - event:  `packet_sent` (Packet sent)
    - time: Sender's current time
    - entity: `"sender"`
    - payload: `{"seq_num": <int>, "bit": <0|1>, "is_retry": <bool>}`
    - Trigger: When Sender completes preparation delay and hands packet to subnet

    - event: `ack_received` (ACK received)
    - time: Sender's current time
    - entity: `"sender"`
    - payload: `{"ack_bit": <0|1>, "is_valid": <bool>}`
    - Trigger: When Sender receives an ACK

    Receiver Events:
    - event: `delay_start` (Receiver starts processing delay)
    - time: Receiver's current time
    - entity: `"receiver"`
    - payload: `{"type": "processing", "duration": <float>}`
    - Trigger: When Receiver starts processing a received packet

    - event: `packet_received` (Packet successfully received)
    - time: Receiver's current time
    - entity: `"receiver"`
    - payload: `{"seq_num": <int>, "bit": <0|1>}`
```

```
  - Trigger: When Receiver completes processing delay and successfully receives packet

  Subnet Events:
  - event: `packet_get` (Packet fate determined)
  - time: Subnet's current time
  - entity: `"subnet"`
  - payload: `{"behavior": <"drop"|"pass">, "channel": <"forward"|"backward">, "noise_value": <int>}`
  - Trigger: When packet arrives at subnet, before transmission delay starts
  - Note: "forward" for Sender->Receiver channel, "backward" for Receiver->Sender channel
```

Listing 3: Operational Specification

F.2 Planning Skeleton

The first major transformation extracts a conceptual DEVS hierarchy from the unstructured text. As shown in Listing 4, the pipeline identifies not only the core physical entities (Sender, Receiver, Subnets) but also intelligently deduces the need for auxiliary utility models, such as `DelayScheduler` and `TimeoutTimer`, to handle the temporal constraints specified in the prompt. This step establishes the structural boundaries before detailed logic is generated.

```
{
  "modules": [
    {
      "name": "ABP_D1",
      "description": "Top-level DEVS model implementing the Alternating Bit Protocol. Data flow (IC): ABP_Sender sends packets to a forward DeterministicLossSubnet, which passes them to ABP_Receiver; ABP_Receiver sends ACKs to a backward DeterministicLossSubnet, which returns them to ABP_Sender. It also couples shared DelayScheduler and TimeoutTimer components.",
      "children_names": [
        "ABP_Sender",
        "ABP_Receiver",
        "DeterministicLossSubnet",
        "DelayScheduler",
        "TimeoutTimer"
      ]
    },
    {
      "name": "ABP_Sender",
      "description": "Coupled model representing the Sender role. Data flow (IC): ABP_SenderController coordinates internal timing by sending requests to DelayScheduler (for preparation) and TimeoutTimer, routing their timeouts back to the controller. EIC/EOC map external packet/ACK flows.",
      "children_names": [
        "ABP_SenderController",
        "DelayScheduler",
        "TimeoutTimer"
      ]
    },
    {
      "name": "ABP_Receiver",
      "description": "Atomic model handling packet reception. Maintains a capacity-1 buffer, undergoes deterministic processing delay, and generates ACK echoes back to the sender.",
      "children_names": []
    },
    {
      "name": "DeterministicLossSubnet",
      "description": "Atomic model simulating a unidirectional network channel. Receives packets/ACKs, determines deterministic drop fate based on noise seed, and outputs surviving packets after a fixed transmission delay.",
      "children_names": []
```

```
    },
    {
      "name": "DelayScheduler",
      "description": "Utility atomic model for scheduling delays. Receives start requests and emits corresponding completion signals after configured sender or receiver delays.",
      "children_names": []
    },
    {
      "name": "TimeoutTimer",
      "description": "Utility atomic model for timeout logic. Receives start/restart requests for outstanding packets and emits a timeout expiration event when the duration elapses.",
      "children_names": []
    },
    {
      "name": "ABP_SenderController",
      "description": "Atomic model executing the core stop-and-wait state machine for the sender. Handles sequence numbers, bit alternation, retransmission logic, and validates ACKs without directly managing the time progression.",
      "children_names": []
    }
  ]
}
```

Listing 4: The skeleton of the ABP_Sender

F.3 Structured PlanTree

Once the skeleton is established, the pipeline performs a top-down refinement to generate the `PlanTree`, transforming conceptual modules into strict structural contracts. Figure 5 illustrates the resulting hierarchical decomposition.

To demonstrate the rigidity of these contracts, Listing 5 provides a condensed view of the `ABP_Sender` node. In this phase, ambiguous natural language is translated into explicitly defined I/O ports, data structures, and state initialization parameters. By strictly defining the schemas (e.g., ensuring `in_ack` only accepts `{'ack_bit': int}`), the `PlanTree` ensures that downstream code generation for each component can occur in parallel without integration conflicts.

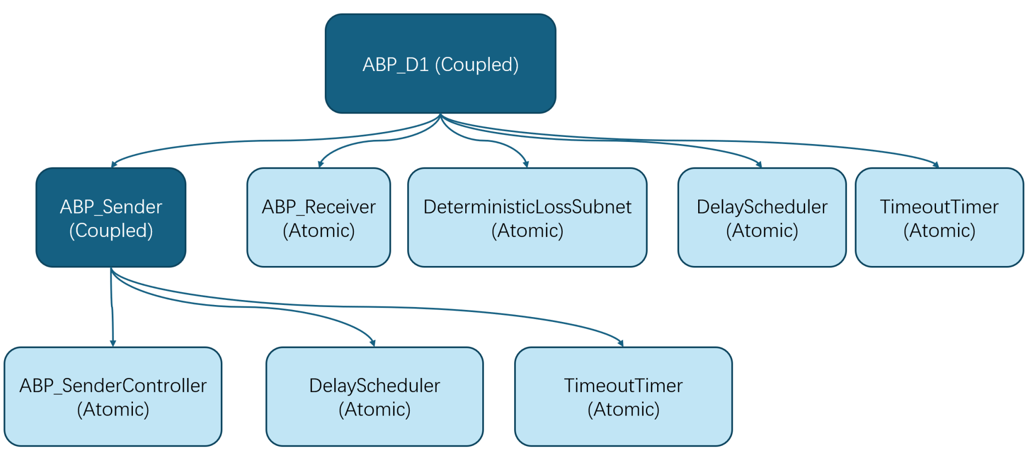

Figure 5: Visualized `PlanTree` hierarchy for the ABP Model. The root model recursively decomposes into sub-models until atomic primitives are reached.

```
{
```

```
  "class_name": "ABP_Sender",
  "plan_phase": {
    "type": "coupled",
    "model_info": {
      "specification": {
        // Functional logic extracted from natural language
        "function": "Implements Sender role... [Details Omitted] ...",

        // Strict Interface Definitions
        "model_init_args": [
          {"name": "total_packets", "type": "int", "structure": "..."},
          {"name": "timeout", "type": "float", "structure": "..."}
        ],

        // Explicit Port Definitions with Protocol
        "input_ports": [
          {
            "name": "in_start",
            "type": "dict",
            "structure": "{'total_packets': int}",
            "protocol": {
              "description": "Optional start command...",
              "initial_signal": "If total_packets>0, auto-start at t=0..."
            }
          },
          {
            "name": "in_ack",
            "type": "dict",
            "structure": "{'ack_bit': int} # ack_bit in {0,1}"
          }
        ],
        "output_ports": [
          {
            "name": "out_data_to_forward",
            "type": "dict",
            "structure": "{'seq_num': int, 'bit': int}"
          }
        ]
      },
      // Architectural Decomposition
      "coupling_specification": "Instantiate children: ABP_SenderController,
   DelayScheduler, TimeoutTimer...",
    }
  },
  // Recursive definition of children
  "children_plan": [
    { "class_name": "ABP_SenderController", ... },
    { "class_name": "DelayScheduler", ... },
    { "class_name": "TimeoutTimer", ... }
  ]
}
```

Listing 5: Fragment of the PlanTree focusing on the ABP_Sender's ModelPlan

F.4 Generated Code Structure

In the final phase, the pipeline compiles the `PlanTree` contracts into executable DEVS Python code. Figure 6 depicts the external and internal coupling graph extracted directly from the generated implementation. The exact correspondence between the JSON-defined ports in Listing 5 and the physical routing in this graph validates the pipeline's ability to maintain architectural consistency from high-level planning down to code synthesis.

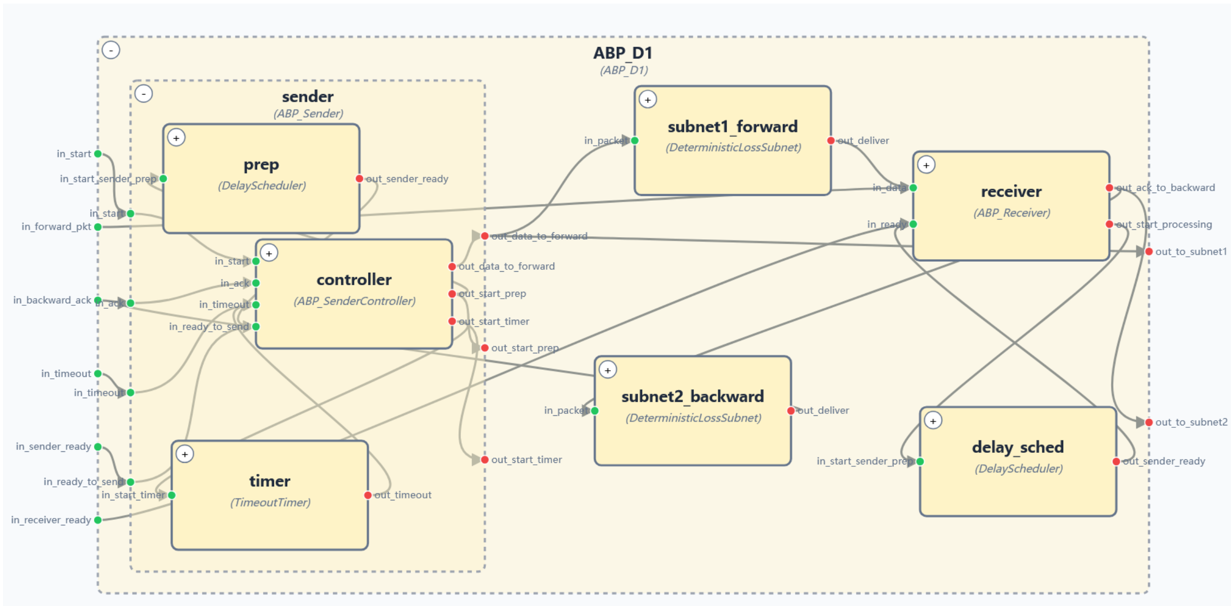

Figure 6: The final connection of the ABP DEVS model.

G Qualitative Analysis: Failure Modes

To better understand the reliability boundaries of different approaches, we manually inspected the failure cases across the benchmark. We observe that DEVS-Gen and iterative agents exhibit fundamentally different failure signatures.

Failures in DEVS-Gen: Semantic Misalignment. Since DEVS-Gen follows a strict "correct-by-construction" structural template, it rarely suffers from the chaotic syntax errors or hallucinatory loops common in standard agents. Instead, its failures are typically local and semantic, stemming from limitations in the underlying LLM's grasp of formal DEVS semantics.

- *Transition Logic inversion:* A recurrent error pattern involves the misinterpretation of the DEVS simulation cycle, specifically the ordering of the output function λ and the internal transition δ_{int}. In the standard DEVS protocol, a model must emit output based on its current state before transitioning to the next state. However, weaker models occasionally attempt to compute output values during the phase of the previous cycle, leading to state-output desynchronization.
- *Protocol Deviations:* In complex scenarios, the model may implement a conceptually correct state machine that misses a specific edge case in the specification, resulting in trace validation failures despite the simulator being structurally sound and executable.

Failures in Iterative Agents: Divergence and Stagnation. In contrast, the failures of baseline agents (SWE-Agent, OpenHands) are often global and operational.

- *Debugging Loops:* When faced with implementation errors, these agents frequently enter "doom loops" where they repeatedly run the code, encounter an error, and attempt superficial fixes (e.g., adding print statements or changing variable names) without resolving the root cause. This explains the high token consumption and timeout rates observed in Table 2.
- *Silent Failures:* A common failure mode for "Lite" baselines is the generation of code that runs without crashing but produces empty or nonsensical event traces due to a lack of understanding of the global event loop driver.

This comparison highlights that while DEVS-Gen is not immune to logic errors, its modularity constrains failures to the component level, making them easier to diagnose and fix compared to the entangled monolithic failures of iterative agents.

H Limitations and Future Work

While our framework provides a verifiable approach to generating discrete-event world models, several limitations define its current scope and present directions for future research:

Scope of Environment Dynamics. As motivated, DEVS-GEN is purposely designed for macro-level, discrete-event environments (e.g., supply chains, service operations, and logical workflows). Consequently, it does not natively model continuous-time differential equations. While this strict discrete scope is sufficient for process-driven domains, some advanced real-world applications—such as smart grids or automated cyber-physical manufacturing—exhibit *hybrid* dynamics, where discrete strategic events intersect with continuous physical evolution. Extending the framework to synthesize Hybrid DEVS models to support these intersecting continuous domains remains a technical direction for future work.

Instruction Adherence and Model Fine-Tuning. The reliability of DEVS-GEN, particularly on smaller models, is bounded by the underlying LLM's instruction-following capabilities. Generation failures frequently stem from minor deviations in output formatting or port naming rather than fundamental logic errors. Because our decoupled workflow decomposes the generation process into standardized, predictable sub-tasks (`ModelPlan`), the prompt distribution is highly stable. Future work can leverage this property by applying Supervised Fine-Tuning (SFT) or Reinforcement Learning from Human Feedback (RLHF) to smaller coding models, which is expected to significantly mitigate formatting failures.

Benchmark Scale and Specification Ambiguity. We introduce a structured benchmark to evaluate simulator synthesis; however, the current dataset relies on a bounded set of scenarios with precise, unambiguous system specifications. This controlled setup is necessary to isolate pure synthesis capability from requirement analysis. In practical deployments, target systems can scale to thousands of interacting components, and user prompts are often incomplete. Future iterations will expand the benchmark to include larger-scale, highly underspecified tasks. Correspondingly, the framework can be enhanced with an interactive elicitation phase to clarify ambiguous requirements, alongside a targeted self-correction loop in Stage 1 to resolve structural topology errors before code synthesis.